

第 1 章 PZ-MCU-TFT7.0-SSD1963 模块简介

本章将教大家如何在普中 STM32 开发板上使用 PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块。对于其他 STM32 开发板，同样也可以参考本文档进行学习，大同小异。本文档分为如下几部分：

- 1.1 PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块简介
- 1.2 硬件连接
- 1.3 软件实现

1.1 PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块简介

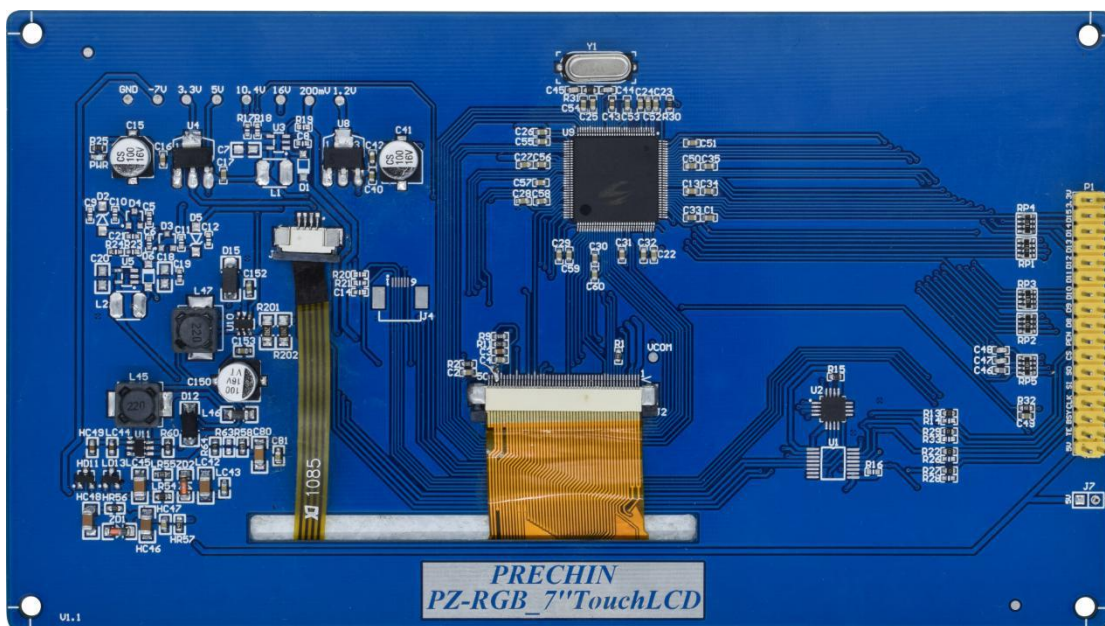
PZ-MCU-TFT7.0-SSD1963 模块是普中推出的一款高性能 7 寸电阻触摸屏模块。该模块屏幕分辨率为 800*480，16 位真彩显示，采用 SSD1963 驱动，该芯片直接自带 GRAM，无需外加驱动器，因而任何单片机，都可以轻易驱动。

模块采用电阻触摸屏，具有非常好的操控效果，非常适合工业应用。模块硬件接口与普中其他液晶模块(2.6' /2.8' /3.5' /4.3' /7' 等)接口完全一致，因而原有产品，硬件上不需要任何变动，只需要稍微修改一下软件，就可以使用我们的 PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块。

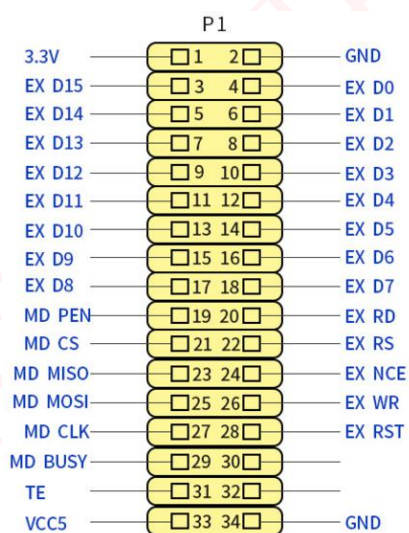
1.1.1 模块引脚说明

PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块通过 2*17 的排针（2.54mm 间距）同外部连接，模块可以与普中的 STM32 开发板直接对接，我们提供相应的例程，用户可以在 STM32 开发板上直接测试。模块外观如下图所示：





触摸屏模块通过 34 (2*17) 个引脚同外部连接，对外接口原理图如下图所示：



序号	名称	说明
1	3.3V	3.3V电源输入引脚
2,34	GND	地线
3-18	EX_D0~D15	双向数据总线
19	MD_PEN	电阻触摸屏,中断信号
20	EX_RD	读使能信号(低电平有效)
21	MD_CS	电阻触摸屏,复位信号
22	EX_RS	命令/数据控制信号(0,命令; 1,数据)
23	MD_MISO	电阻触摸屏,数据输出脚
24	EX_NCE	LCD控制器片选信号(低电平有效)
25	MD_MOSI	电阻触摸屏,数据输入脚
26	EX_WR	写使能信号(低电平有效)
27	MD_CLK	电阻触摸屏时钟信号
28	MX_RST	复位信号(低电平有效)
29	MD_BUSY	NC,电阻触摸屏未用到
30,32	NC	未用到
31	TE	撕裂效应信号
33	5V	5V电源输入引脚

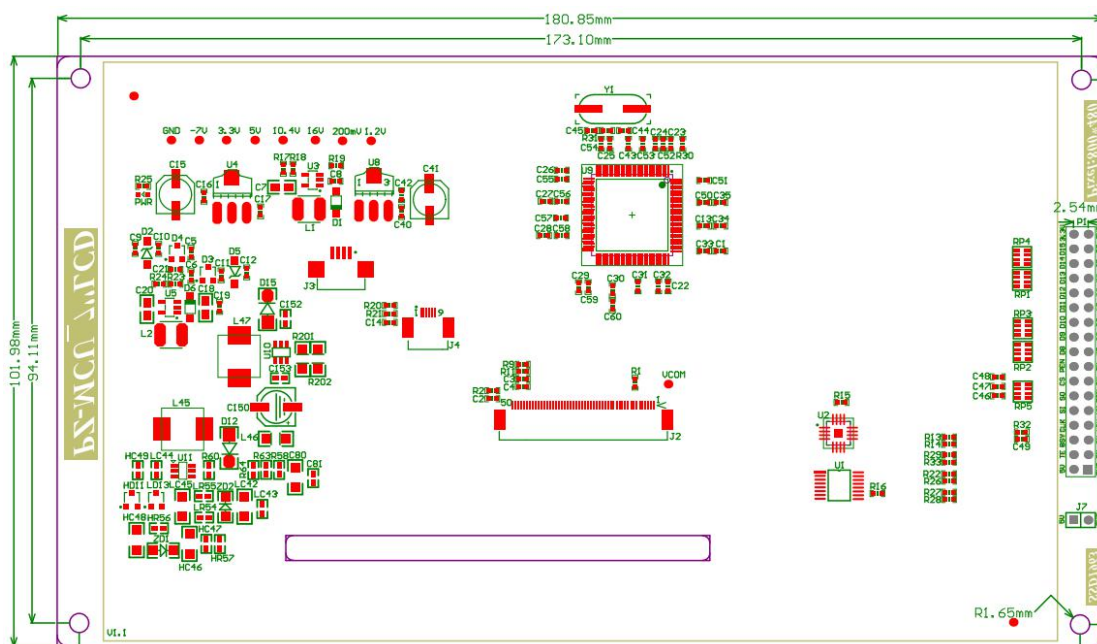
从上图可以看出，在 16 位模式下：LCD 控制器总共需要 21 个 IO 口驱动（不包括 TE 信号），电阻触摸屏需要 5 个 IO 口驱动，这样整个模块需要 26 个 IO 口驱动。而在 8 位/9 位/12 位模式下，则可以少用一些 IO，不过驱动速度就会相应的下降。

TE 是撕裂效应信号，是 LCD 控制器反馈给单片机的信号，用于指示 LCD 控制器的显示状态。在非显示周期内，TE 信号为高。因此，本信号使单片机通过观察非显示周期发送数据，以避免撕裂。不过这个信号我们一般用不到，一般情况下可以不需要理会此信号。

特别注意：模块需要 5V 电源供电才可以正常工作，5V 电源用于背光供电，通过稳压芯片获得的 3.3V 用于除背光外的其他电源部分供电。

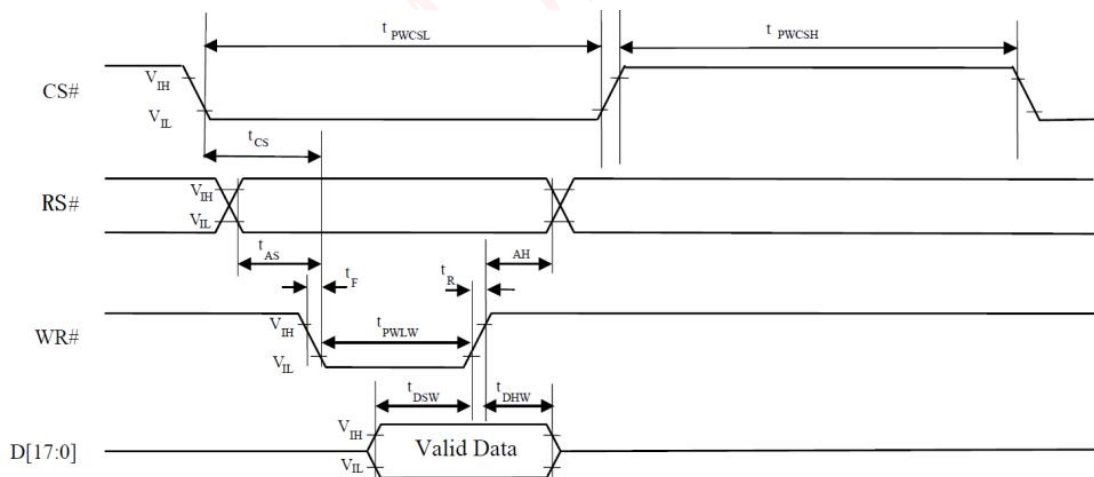
1.1.2 接口尺寸

PZ-MCU-TFT7.0-SSD1963 电阻触摸屏模块尺寸结构图如下：

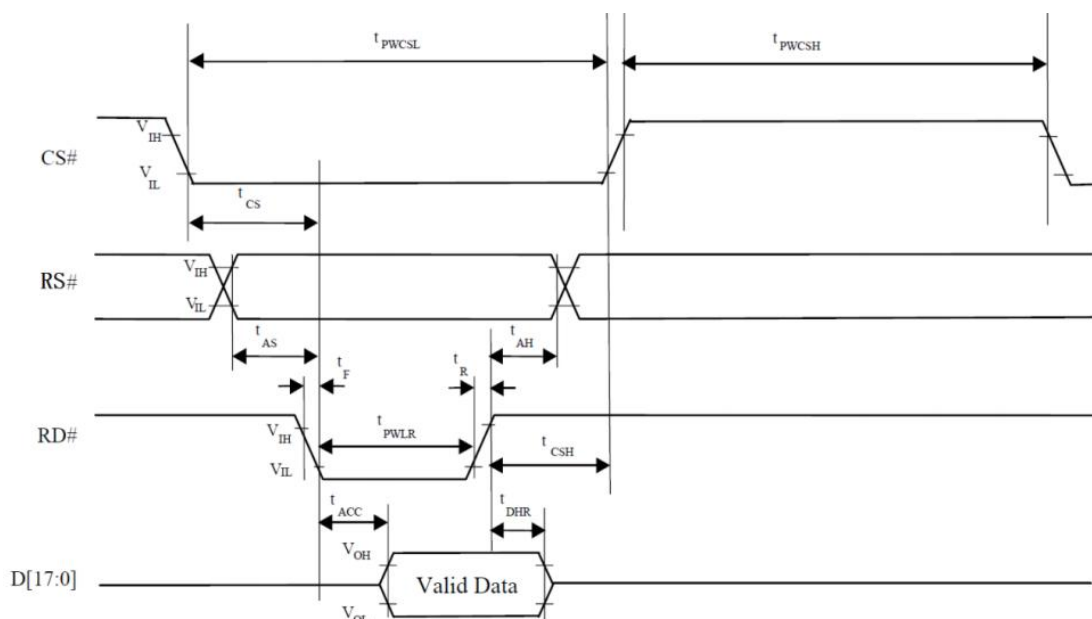


1.1.3 模块接口时序

模块采用 SSD1963 作为驱动器，支持 8 位/9 位/12 位/16 位 8080 总线接口，总线写时序如下图所示：



图中，当 RS 为 0 的时候，表示写入的是寄存器地址（0~7），RS 为 1 的时候，表示写入的是数据（寄存器值/GRAM 数据）。总线读时序如下图所示：



Symbol	Parameter	Min	Typ	Max	Unit
f_{MCLK}	System Clock Frequency*	1	-	110	MHz
t_{MCLK}	System Clock Period*	$1/f_{MCLK}$	-	-	ns
t_{PWCSL}	Control Pulse High Width Write Read	13 30	$1.5 * t_{MCLK}$ $3.5 * t_{MCLK}$	-	ns
t_{PWCSH}	Control Pulse Low Width Write (next write cycle) Write (next read cycle) Read	13 80 80	$1.5 * t_{MCLK}$ $9 * t_{MCLK}$ $9 * t_{MCLK}$	-	ns
t_{AS}	Address Setup Time	1	-	-	ns
t_{AH}	Address Hold Time	2	-	-	ns
t_{DSW}	Write Data Setup Time	4	-	-	ns
t_{DHW}	Write Data Hold Time	1	-	-	ns
t_{PWLW}	Write Low Time	12	-	-	ns
t_{DHR}	Read Data Hold Time	1	-	-	ns
t_{ACC}	Access Time	32	-	-	ns
t_{PWLr}	Read Low Time	36	-	-	ns
t_R	Rise Time	-	-	0.5	ns
t_F	Fall Time	-	-	0.5	ns
t_{CS}	Chip select setup time	2	-	-	ns
t_{CSH}	Chip select hold time to read signal	3	-	-	ns

* System Clock denotes external input clock (PLL-bypass) or internal generated clock (PLL-enabled)

一般我们设置 f_{MCLK} 的频率为 100MHz，因此写周期长度最小为 3 个 t_{MCLK} ，即写一个像素，最快需要 3 个 t_{MCLK} ，即：30ns，对于本模块，屏幕分辨率是 800*480，因此可以计算得出：频率的刷新率，最快是 86.8 帧。模块的读取速度相对较慢：读周期需要 12.5 个 t_{MCLK} ，即：125ns。

有关 SSD1963 内寄存器指令，可参考资料内 SSD1963 数据手册。

1.2 硬件连接

见后面章节实验。（后面实验章节由普中 STM32 开发攻略筛选）

1.3 软件实现

见后面章节实验。（后面实验章节由普中 STM32 开发攻略筛选）

2 其他

（1）购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

（2）资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

（3）技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-36564227（转技术）

第 2 章 FSMC-TFTLCD 显示实验

以前我们用 51 单片机控制 TFTLCD（TFT 液晶屏）时，使用的是 IO 口模拟 80 接口时序。这一章我们来学习如何使用 STM32F1 的 FSMC（灵活的静态存储控制器）接口控制 TFTLCD 显示。TFTLCD 可以显示 16 位色真彩图片。本章要实现的功能是：在 TFTLCD 上显示 ASCII 字符和汉字，同时 D1 指示灯闪烁，提示系统正常运行。学习本章可以参考《STM32F10x 中文参考手册》-19 灵活的静态存储控制器（FSMC）章节，特别是寄存器介绍部分。若结合视频学习效果更佳。本章分为如下几部分内容：

2.1 TFTLCD 和 FSMC 介绍

2.2 FSMC 配置步骤

2.3 硬件设计

2.4 软件设计

2.5 实验现象

2.1 TFTLCD 和 FSMC 介绍

2.1.1 TFTLCD 简介

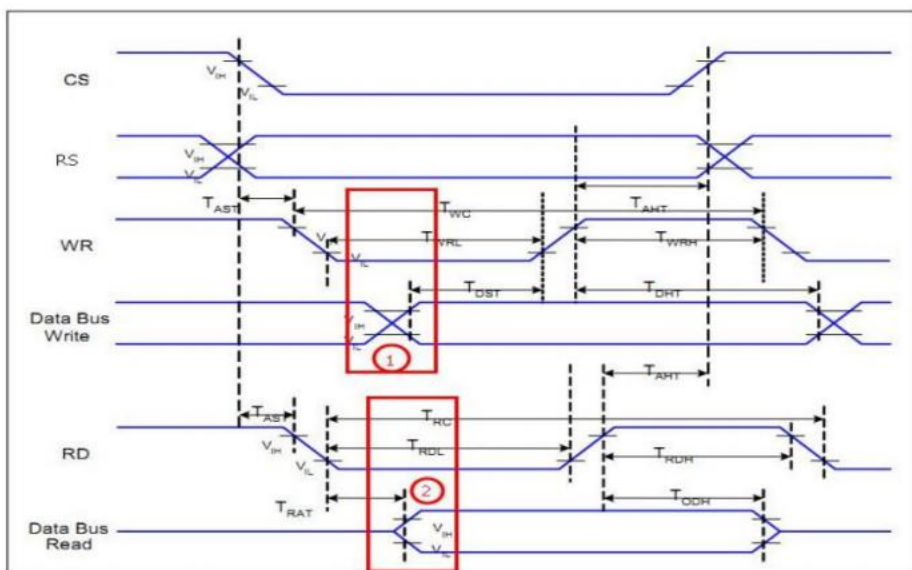
TFT-LCD 是薄膜晶体管液晶显示器英文 thin film transistor-liquid crystal display 字头的缩写。TFT 液晶为每个像素都设有一个薄膜晶体管(TFT)，每个像素都可以通过点脉冲直接控制，因而每个节点都相对独立，并可以连续控制，不仅提高了显示屏的反应速度，同时可以精确控制显示色阶，所以 TFT 液晶的色彩更真，因此 TFT-LCD 也被叫做真彩液晶显示器。

常用的 TFT 液晶屏接口有很多种，8 位、9 位、16 位、18 位都有，这里的位数表示的是彩屏数据线的数量。常用的通信模式主要有 6800 模式和 8080 模式，对于 TFT 彩屏通常都使用 8080 并口（简称 80 并口）模式。

如果大家接触过 LCD1602 或者 LCD12864 等，那么就会发现 8080 模式的读写时序其实跟 LCD1602 或者 LCD12864 的读写时序是差不多的。8080 接口有 5 条基本的控制线和多条数据线，数据线的数量主要看液晶屏使用的是几位模式，有 8 根、9 根、16 根、18 根四种类型。它们的功能如下：

RST	0: 复位选择 1: 取消复位
CS	0: 片选选择 1: 取消片选
RS	0: 控制寄存器 1: 数据寄存器
RD	0: 读选择 1: 读取消
WR	0: 写选择 1: 写取消
DB0~DB8	数据线

接下来我们来看一下 8080 接口模式的时序，如下图：



从上图我们就可以很清晰的看得出液晶屏的读写时序：

①：在 WR 跳变为低电平之后，液晶屏开始读取总线上面的数据。如果使用 I/O 口模拟写入的时候，可以先在总线上写入数据，然后在跳变 WR，以保证当读取的时候，总线上面的数据是稳定的。

②：在 RD 跳变为低电平之后，液晶屏放置数据到总线上面。

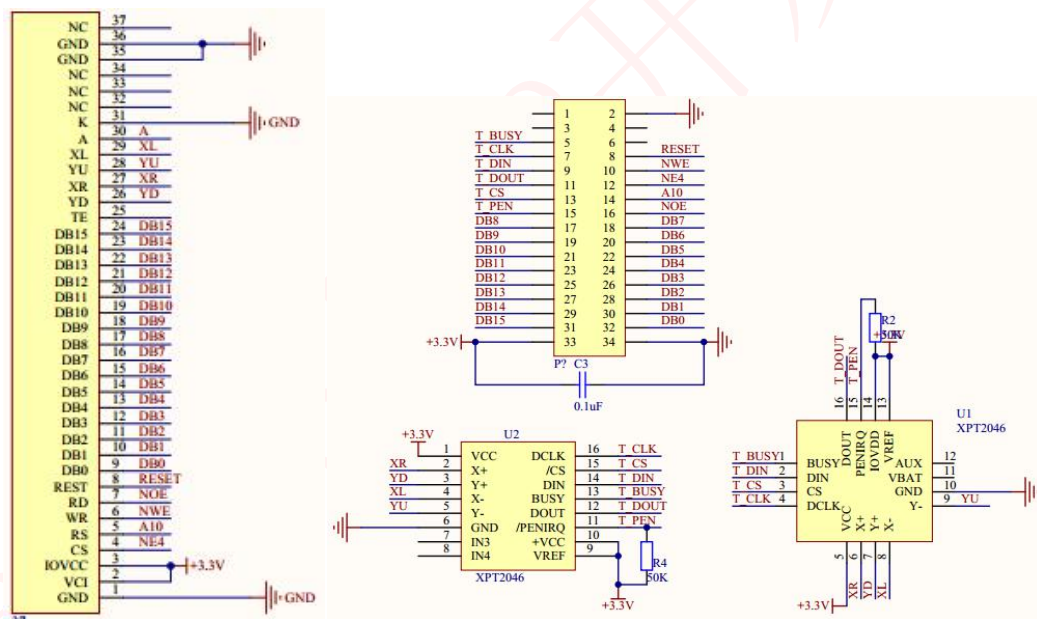
像以前我们使用 51 单片机就是通过单片机的 I/O 口模拟 8080 时序进行 TFT 彩屏控制，但是对于我们 STM32F767 就不需要这样模拟了，我们 STM32F767 自带了 FMC 接口，这个后面我们会介绍到。

下面我们来介绍下 TFTLCD 模块，我们公司推出的 TFTLCD 模块有很多种，按照屏幕大小的不同可分为 2.0、2.4、2.8、3.0、3.2、3.5、3.6、4.3、4.5、7 寸等，不同尺寸的彩屏对应的分辨率可能不同，比如说 3.5 寸的彩屏分辨率为 320*480（宽*高），4.5 寸的为 480*854，当然这个具体要看对应彩屏的数据手册，彩屏数据手册在光盘的“\2—开发板原理图\彩屏原理图”。按照 TFT 彩屏驱动芯片的不同可分为海信 HX83xx、ILI93xx、R615xx、LG45xx、NT355xx 等等，你手上的彩屏驱动芯片具体是哪一种，需要看下彩屏正面左上角或背面型号，通常我们都会将彩屏的驱动芯片型号放在 TFTLCD 模块的左上角或背面。我们的 TFTLCD 模块都自带触摸功能，可用来做输入控制。

本章我们就以 3.5 寸的 TFTLCD 模块为例来介绍（其他尺寸的彩屏和驱动芯片使用方法类似），该模块驱动芯片型号是 HX8357D，分辨率为 320*480，接口为 16 位的 80 并口，自带触摸功能。该模块的外观图如下图所示：



该模块原理图如下图所示：



TFTLCD 模块采用 2*17 的 2.54 公排针与外部连接，从上图可以看出，此 TFTLCD 模块采用 16 位的并口方式与外部连接，之所以不采用 8 位的方式，是因为彩屏的数据量比较大，尤其在显示图片的时候，如果用 8 位数据线，就会比 16 位方式慢一倍以上，我们当然希望速度越快越好，所以选择 16 位的接口，当然有的 TFTLCD 本身就是 8 位数据线。图中还列出了触摸屏芯片的接口，关于触摸屏本章我们不多介绍，在后面的章节会有详细的介绍。该模块的 80 并口有如下一些信号线：

CS： TFTLCD 片选信号。

WR: 向 TFTLCD 写入数据控制。

RD: 从 TFTLCD 读取数据控制。

RS: 命令/数据选择 (0, 读写命令; 1, 读写数据)。

DB[15: 0]: 16 位双向数据线。

RST: TFTLCD 复位。

80 并口的通信时序前面已经介绍,这里需要说明的是,TFTLCD 模块的 RST 信号线是直接接到 STM32F1 的复位脚上,并不由软件控制,这样可以节省一个 IO 口。所以要控制 TFTLCD 模块显示,总共需要 20 个 IO 口 (除触摸功能管脚)。

知道了模块的管脚功能及通信时序,接下来我们就来介绍下如何让液晶模块显示。通常按照以下几步即可实现 TFT 液晶显示:

(1) 设置 STM32F1 与 TFTLCD 模块相连接的 IO

要让 TFTLCD 模块显示,首先得初始化 TFTLCD 模块与 STM32F1 相连的 IO 口,以便控制 TFTLCD。这里我们用使用的是 STM32F1 的 FSMC,FSMC 将在后面小节向大家详细介绍。

(2) 初始化 TFTLCD 模块 (写入一系列设置值)

初始化 IO 口,接着就是对 TFTLCD 进行配置,首先就是要复位下 LCD,由于模块的复位引脚是接在 STM32F1 复位上的,所以直接按下开发板复位键即可,然后就是初始化序列,即向 LCD 控制器写入一系列的设置值 (比如 RGB 格式、LCD 显示方向、伽马校准等),这部分代码一般 LCD 厂商会提供,我们直接使用这些初始化序列即可,无需深入研究。关于这些设置值可以在你所使用的彩屏模块驱动芯片数据手册内查找到,只不过这些数据手册全是英文的,其实也不是很难,我们用到的只是几个设置值而已,不认识的可以百度翻译下。初始化完成之后,LCD 就可以正常使用了。

(3) 将要显示的内容写到 TFTLCD 模块内

这一步需要按照:设置坐标→写 GRAM 指令→写 GRAM 来实现,但是这个步骤,只是一个点的处理,如果我们想要显示字符或数字,就必须多次使用这个步骤,从而达到显示字符或数字的目的,一般我们会设计一个函数来封装这些过程 (实现字符或数字的显示),之后只需调用该函数,就可以实现字符或数字的显示了。

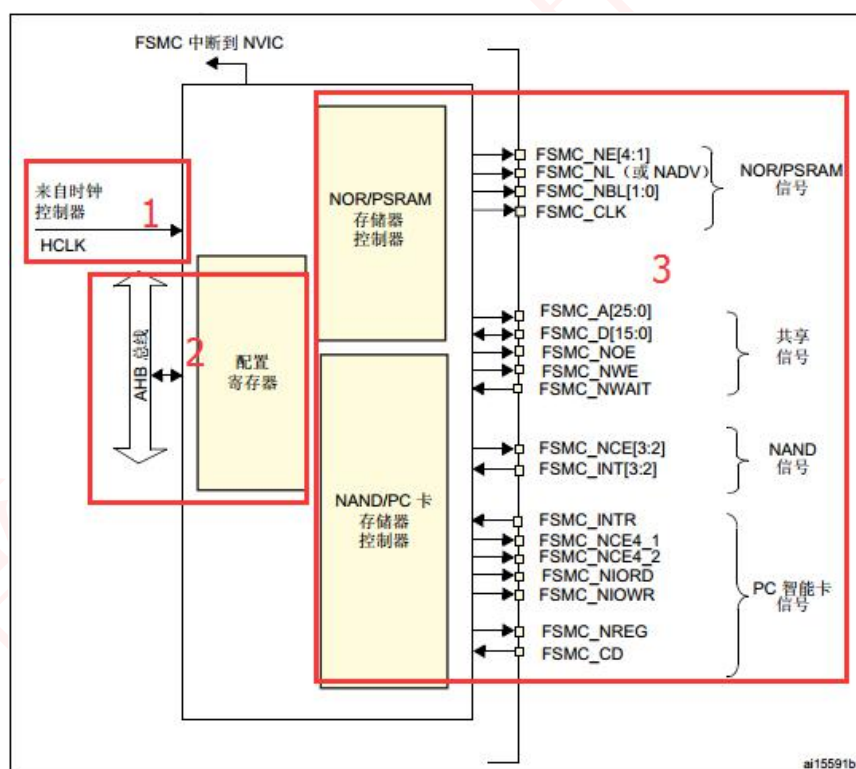
这一部分内容等到我们后面编写程序的时候大家就可以看到，其实还是比较简单的。接下来我们就来揭开 STM32F1 的 FSMC 的神秘面纱。

2.1.2 FSMC 简介

STM32F103 100 引脚以上系列芯片都带有 FSMC 接口，我们开发板上使用的是 STM32F103ZET6，因此也具有 FSMC 接口。

FSMC(Flexible Static Memory Controller，灵活的静态存储控制器)是 STM32 系列采用一种新型的存储器扩展技术，能够连接同步、异步存储器和 16 位 PC 存储卡。STM32 通过 FSMC 可以与 SRAM、ROM、PSRAM、NOR Flash 和 NANDFlash 等存储器的引脚直接相连。

STM32F1 的 FSMC 内部框图如下图所示：（大家也可以查看《STM32F10x 中文参考手册》-19 灵活的静态存储控制器（FSMC）-19.2 章节内容）



我们把 FMC 结构框图分成 3 个子模块，按照顺序依次进行简单介绍。

(1) 标号 1：时钟输入

FSMC 的时钟来至时钟控制器 HCLK，在前面我们讲解“存储器与寄存器”时，我们知道，AHB 区域内包含 FSMC 模块，所以如果要使用 FSMC，必须使能 AHB 总

线时钟。

(2) 标号 2: AHB 接口

CPU 和其它 AHB 总线主设备可通过该 AHB 从设备接口访问外部静态存储器。FSMC 可通过一个寄存器组进行配置。有关 NOR Flash/PSRAM 控制寄存器的详细说明,请参《STM32F10x 中文参考手册》-19 灵活的静态存储控制器 (FSMC) -19.5 章节内容。

(3) FSMC 外部设备

STM32F1 的 FSMC 将外部设备分为 2 类: NOR/PSRAM 设备、NAND/PC 卡设备。他们共用地址数据总线等信号,但具有不同的 CS 以区分不同的设备。

本章实验我们使用的是 FSMC 的 NOR/PSRAM 存储器控制器部分,即把 TFTLCD 当成 SRAM 设备使用。为什么可以把 TFTLCD 当成 SRAM 设备用,这个首先要了解 NOR/PSRAM 存储器控制器的接口信号,其接口信号功能如下:

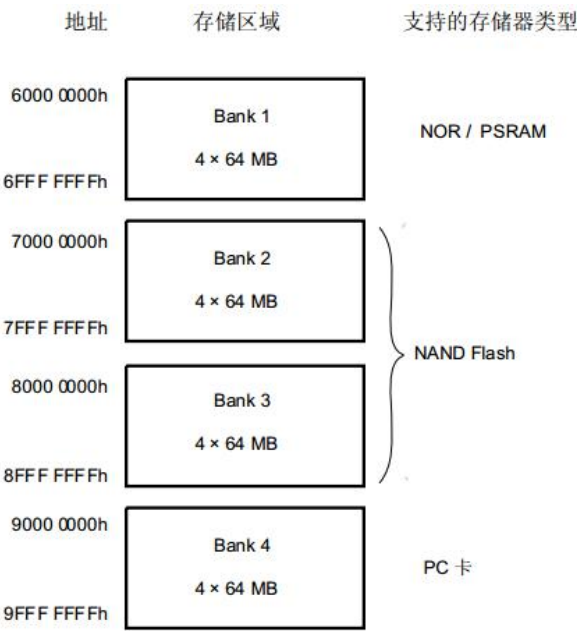
FSMC 信号名称	I/O	功能
CLK	O	时钟 (仅用于 PSRAM 同步突发)
A[25:0]	O	地址总线
D[15:0]	I/O	数据双向总线
NE[x]	O	片选, $x = 1..4$ (在 PSRAM 应用中被称作 NCE (Cellular RAM, 即 CRAM))
NOE	O	输出使能
NWE	O	写入使能
NL(= NADV)	O	仅用于 PSRAM 输入的地址有效信号 (存储器信号名称: NADV)
NWAIT	I	PSRAM 发送给 FSMC 的等待输入信号
NBL[1]	O	高字节使能 (存储器信号名称: NUB)
NBL[0]	O	低字节使能 (存储器信号名称: NLB)

从上图中可以看出外部 SRAM 的控制一般有: 地址线 (如 A0~A25)、双向数据线 (如 D0~D15)、写信号 (NWE)、读信号 (NOE)、片选信号 (NE[x]), 如果 SRAM 支持字节控制, 那么还有 UB/LB 信号。而 TFTLCD 的信号我们在上一节有介绍, 包括: RS、DB0-DB15、WR、RD、CS、RST 等, 其中真正在操作 LCD 的时候需要用到的就只有: RS、DB0-DB15、WR、RD、CS。这样一来它们的操作接口信号完全类似, 唯一不同就是 TFTLCD 有 RS 信号, 但是没有地址信号。

TFTLCD 通过 RS 信号来决定传送的数据是数据还是命令, 本质上可以理解为一个地址信号, 比如我们把 RS 接在 A0 上面, 那么当 FSMC 控制器写地址 0 的时候, 会使得 A0 变为 0, 对 TFTLCD 来说, 就是写命令。而 FSMC 写地址 1

的时候，A0 将会变为 1，对 TFTLCD 来说，就是写数据了。这样，就把数据和命令区分开了，他们其实就是对应 SRAM 操作的两个连续地址。当然 RS 也可以接在其他地址线上，我们 STM32F1 开发板是把 RS 连接在 A10 上面的。

知道了可以将 TFTLCD 当做 SRAM 设备用，下面我们就来看下 FSMC 的外部设备地址映射，从 FSMC 的角度，外部存储器被划分为 4 个固定大小的存储区域（Bank），每个存储区域的大小为 256 MB，共 1GB 空间。如下图所示：



本章实验使用到的是 Bank1，所以我们只讲解这块存储区域，其他的区域大家可参考《STM32F10x 中文参考手册》-19 灵活的静态存储控制器（FSMC）章节内容。

存储区域 1 可连接多达 4 个 NOR Flash 或 PSRAM 存储器器件。此存储区域被划分为 4 个 NOR/PSRAM 区域，带 4 个专用片选信号。存储区域 2 和 3 用于连接 NAND Flash 器件（每个存储区域一个器件）。存储区域 4 用于连接 PC 卡设备。对于每个存储区域，所要使用的存储器类型由用户在配置寄存器中定义。STM32F1 的 FSMC 各 Bank 配置寄存器如下图所示：

内部控制器	存储块	管理的地址范围	支持的设备类型	配置寄存器
NOR FLASH 控制器	Bank1	0X6000, 0000~ 0X6FFF, FFFF	SRAM/ROM NOR FLASH PSRAM	FSMC_BCR1/2/3/4 FSMC_BTR1/2/2/3 FSMC_BWTR1/2/3/4
NAND FLASH /PC CARD 控制器	Bank2	0X7000, 0000~ 0X7FFF, FFFF	NAND FLASH	FSMC_PCR2/3/4 FSMC_SR2/3/4 FSMC_PMEM2/3/4 FSMC_PATT2/3/4
	Bank3	0X8000, 0000~ 0X8FFF, FFFF		FSMC_PIO4
	Bank4	0X9000, 0000~ 0X9FFF, FFFF	PC Card	FSMC_ECCR2/3

STM32F1 的 FSMC 存储块 1(Bank1)又被分为 4 个区,每个区管理 64M 字节空间,每个区都有独立的寄存器对所连接的存储器进行配置。Bank1 的 256M 字节空间由 28 根地址线(HADDR[27:0])寻址。这里 HADDR 是内部 AHB 地址总线,其中 HADDR[25:0] 来自外部存储器地址 FSMC_A[25:0],而 HADDR[26:27]对 4 个区进行寻址。如下图所示:

Bank1 所选区	片选信号	地址范围	HADDR	
			[27:26]	[25:0]
第 1 区	FSMC_NE1	0X6000, 0000~63FF, FFFF	00	FSMC_A[25:0]
第 2 区	FSMC_NE2	0X6400, 0000~67FF, FFFF	01	
第 3 区	FSMC_NE3	0X6800, 0000~6BFF, FFFF	10	
第 4 区	FSMC_NE4	0X6C00, 0000~6FFF, FFFF	11	

本章实验我们使用的是 Bank1 的第 4 区,即起始地址为 0X6C000000。这里要特别注意 HADDR[25:0], HADDR[25:0]包含外部存储器地址。由于 HADDR 为字节地址,而存储器按字寻址,所以根据存储器数据宽度不同,实际向存储器发送的地址也将有所不同,如下图所示:

存储器宽度 ⁽¹⁾	向存储器发出的数据地址	最大存储器容量(位)
8 位	HADDR[25:0]	64 MB x 8 = 512 Mb
16 位	HADDR[25:1] >> 1	64 MB/2 x 16 = 512 Mb

如果外部存储器的宽度为 8 位,FSMC 将使用内部的 HADDR[25:0] 地址来作为对外部存储器的寻址地址 FSMC_A[25:0]。

这里请大家特别留意,如果外部存储器的宽度为 16 位,FSMC 将使用内部的 HADDR[25:1] 地址来作为对外部存储器的寻址地址 FSMC_A[24:0],相当于右移了一位,在后面我们设置 A10 地址的时候就要使用到。无论外部存储器的宽度为 16 位还是 8 位,FSMC_A[0] 都应连接到外部存储器地址 A[0]。

另外，HADDR[27:26]的设置，是不需要我们干预的，比如：当你选择使用 Bank1 的第三个区，即使用 FSMC_NE3 来连接外部设备的时候，即对应了 HADDR[27:26]=10，我们要做的就是配置对应第 3 区的寄存器组，来适应外部设备即可。FSMC 的各 Bank 配置寄存器在上图已列出。

对于 NOR FLASH 控制器，主要是通过 FSMC_BCRx、FSMC_BTRx 和 FSMC_BWTRx 寄存器设置（其中 x=1~4，对应 4 个区）。通过这 3 个寄存器，可以设置 FSMC 访问外部存储器的时序参数，拓宽了可选用的外部存储器的速度范围。

FSMC 的 NOR FLASH 控制器支持同步和异步突发两种访问方式。选用同步突发访问方式时，FSMC 将 HCLK(系统时钟)分频后，发送给外部存储器作为同步时钟信号 FSMC_CLK。此时需要的设置的时间参数有 2 个：

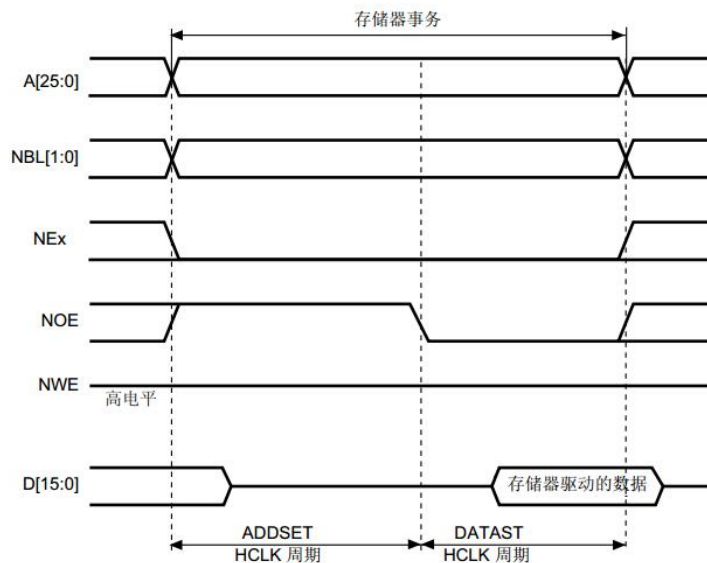
- ①HCLK 与 FSMC_CLK 的分频系数(CLKDIV)，可以为 2~16 分频；
- ②同步突发访问中获得第 1 个数据所需要的等待延迟(DATLAT)。

对于异步突发访问方式，FSMC 主要设置 3 个时间参数：地址建立时间(ADDSET)、数据建立时间(DATAST)和地址保持时间(ADDHLD)。FSMC 综合了 SRAM / ROM、PSRAM 和 NORFlash 产品的信号特点，定义了 4 种不同的异步时序模型。选用不同的时序模型时，需要设置不同的时序参数，如下图所示：

时序模型		简单描述	时间参数
异步	Model	SRAM/CRAM 时序	DATAST、ADDSET
	ModeA	SRAM/CRAM OE 选通型时序	DATAST、ADDSET
	Mode2/B	NOR FLASH 时序	DATAST、ADDSET
	ModeC	NOR FLASH OE 选通型时序	DATAST、ADDSET
	ModeD	延长地址保持时间的异步时序	DATAST、ADDSET、ADDHLK
同步突发		根据同步时钟 FSMC_CLK 读取多个顺序单元的数据	CLKDIV、DATLAT

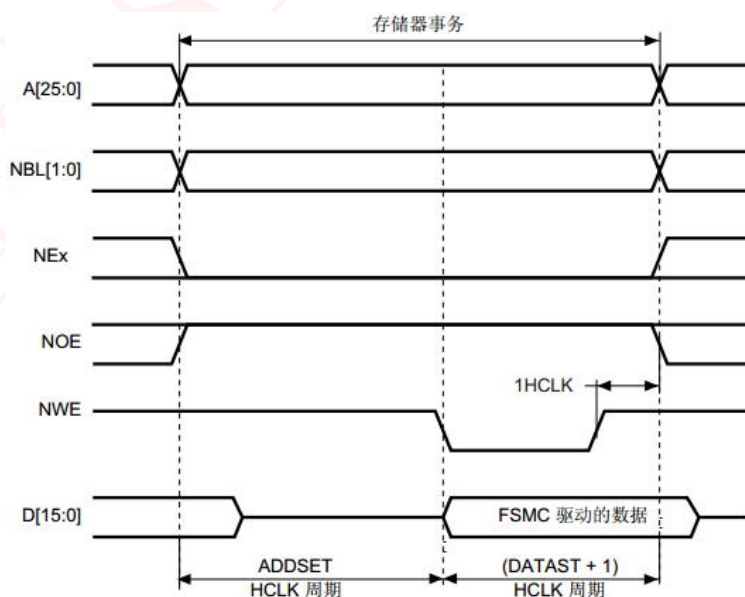
在实际扩展时，根据选用存储器的特征确定时序模型，从而确定各时间参数与存储器读 / 写周期参数指标之间的计算关系；利用该计算关系和存储芯片数据手册中给定的参数指标，可计算出 FSMC 所需要的各时间参数，从而对时间参数寄存器进行合理的配置。

本章实验我们使用异步模式 A (ModeA) 方式来控制 TFTLCD，模式 A 的读操作时序如下图所示：



模式 A 支持独立的读写时序控制,这个对我们驱动 TFTLCD 来说非常有用,因为 TFTLCD 在读的时候,一般比较慢,而在写的时候可以比较快,如果读写用一样的时序,那么只能以读的时序为基准,从而导致写的速度变慢,或者在读数据的时候,重新配置 FSMC 的延时,在读操作完成的时候,再配置回写的时序,这样虽然也不会降低写的速度,但是频繁配置,比较麻烦。而如果有独立的读写时序控制,那么我们只要初始化的时候配置好,之后就不用再配置,既可以满足速度要求,又不需要频繁改配置。

模式 A 的写操作时序如下图所示:



模式 A 读写时序中的 ADDSET 与 DATAST,是通过不同的寄存器设置的。由

于篇幅限制,本章并没有对 FSMC 相关寄存器进行介绍,大家可以参考《STM32F7xx 中文参考手册》-13 可变存储控制器 (FMC) 章节寄存器内容,里面有详细的讲解。不过,这里还要给大家做下科普,在 KEIL 的寄存器定义里面,并没有定义 FMC_BCRx、FMC_BTRx、FMC_BWTRx 等这个单独的寄存器,而是将他们进行了一些组合。

FSMC_BCRx 和 FSMC_BTRx,组合成 BTCR[8] 寄存器组,他们的对应关系如下:

BTCR[0]对应 FSMC_BCR1, BTCR[1]对应 FSMC_BTR1

BTCR[2]对应 FSMC_BCR2, BTCR[3]对应 FSMC_BTR2

BTCR[4]对应 FSMC_BCR3, BTCR[5]对应 FSMC_BTR3

BTCR[6]对应 FSMC_BCR4, BTCR[7]对应 FSMC_BTR4

FSMC_BWTRx 则组合成 BWTR[7], 他们的对应关系如下:

BWTR[0]对应 FSMC_BWTR1, BWTR[2]对应 FSMC_BWTR2,

BWTR[4]对应 FSMC_BWTR3, BWTR[6]对应 FSMC_BWTR4,

BWTR[1]、BWTR[3]和 BWTR[5]保留,没有用到。

FSMC 内部还是比较复杂的,如果看不懂的可以暂时放下,因为我们使用的是库函数开发,只需简单配置下即可使用。

2.2 FSMC 配置步骤

接下来我们介绍下如何使用库函数对 FSMC 进行配置。这个也是在编写程序中必须要了解的。具体步骤如下: (FSMC 相关库函数在 stm32f10x_fsmc.c 和 stm32f10x_fsmc.h 文件中)

(1) FSMC 初始化

FSMC 的初始化主要是配置 FSMC_BCRx, FSMC_BTRx, FSMC_BWTRx 这三个寄存器,固件库内提供了 3 个初始化函数对这些寄存器配置。FSMC 初始化库函数如下:

```
FSMC_NORSRAMInit();
```

```
FSMC_NANDInit();
```

```
FSMC_PCCARDInit();
```

这三个函数分别用来初始化 4 种类型存储器。这里根据名字就很好判断对

应关系。用来初始化 NOR 和 SRAM 使用同一个函数 `FSMC_NORSRAMInit()`。所以我们之后使用的 FSMC 初始化函数为 `FSMC_NORSRAMInit()`。该初始化函数原型是：

```
void FSMC_NORSRAMInit(FSMC_NORSRAMInitTypeDef*  
FSMC_NORSRAMInitStruct);
```

这个函数只有一个参数，是一个结构体指针变量，结构体类型是 `FSMC_NORSRAMInitTypeDef`，其内成员变量非常多，因为 FSMC 相关的配置项非常多。下面我们简单介绍下它的成员：

```
1.  typedef struct  
2.  {  
3.      uint32_t FSMC_Bank;  
4.      uint32_t FSMC_DataAddressMux;  
5.      uint32_t FSMC_MemoryType;  
6.      uint32_t FSMC_MemoryDataWidth;  
7.      uint32_t FSMC_BurstAccessMode;  
8.      uint32_t FSMC_AsynchronousWait;  
9.      uint32_t FSMC_WaitSignalPolarity;  
10.     uint32_t FSMC_WrapMode;  
11.     uint32_t FSMC_WaitSignalActive;  
12.     uint32_t FSMC_WriteOperation;  
13.     uint32_t FSMC_WaitSignal;  
14.     uint32_t FSMC_ExtendedMode;  
15.     uint32_t FSMC_WriteBurst;  
16.     FSMC_NORSRAMTimingInitTypeDef* FSMC_ReadWriteTimingStruct;  
17.     FSMC_NORSRAMTimingInitTypeDef* FSMC_WriteTimingStruct;  
18. }FSMC_NORSRAMInitTypeDef;
```

从这个结构体我们可以看出，前面有 13 个基本类型 (`uint32_t`) 的成员变量，这 13 个参数是用来配置片选控制寄存器 `FSMC_BCRx`。最后面还有两个 `FSMC_NORSRAMTimingInitTypeDef` 指针类型的成员变量。前面我们讲到，FSMC 有读时序和写时序之分，所以这里就是用来设置读时序和写时序的参数了，也就是说，这两个参数是用来配置寄存器 `FSMC_BTRx` 和 `FSMC_BWTRx`，后面我们会讲解到。下面我们就来看看这些成员：

FSMC_Bank：用来设置使用到的存储块标号和区号，本章实验我们是使用的存储块 1 区号 4，所以选择值为 `FSMC_Bank1_NORSRAM4`。

FSMC_DataAddressMux：用于配置 FSMC 的数据线与地址线是否复用。FSMC

支持数据与地址线复用或非复用两种模式。在非复用模式下 16 位数据线及 26 位地址线分开始用；复用模式则低 16 位数据/地址线复用，仅对 NOR 和 PSRAM 有效。在复用模式下，推荐使用地址锁存器以区分数据与地址。本实验使用 FSMC 模拟 8080 时序，仅使用一根地址线 A10 提供 8080 的 RS 信号，所以不需要复用，即设置为 `FSMC_DataAddressMux_Disable`。

FSMC_MemoryType: 用来设置 FSMC 外接的存储器类型，可选类型为 NOR FLASH 模式、PSARM 模式及 SRAM 模式。我们这里把 TFTLCD 当做 SRAM 使用，所以选择值为 `FSMC_MemoryType_SRAM`。

FSMC_MemoryDataWidth: 用来设置 FSMC 接口的数据宽度，可选择 8 位还是 16 位，这里我们是 16 位数据宽度，所以选择值为 `FSMC_MemoryDataWidth_16b`。

FSMC_WriteOperation: 用于配置写操作使能，如果禁止了写操作，FSMC 不会产生写时序，但仍可从存储器中读出数据。本实验需要向 TFTLCD 内写数据，所以要写使能，配置为 `FSMC_WriteOperation_Enable`(写使能)。

FSMC_ExtendedMode: 用于配置是否使用扩展模式，在扩展模式下，读时序和写时序可以使用独立时序模式。如读时序使用模式 A，写时序使用模式 B，这些 A、B、C、D 模式实际上差别不大，主要是在使用数据/地址线复用的情况下，FSMC 信号产生的时序不一样。

FSMC_BurstAccessMode: 用于配置访问模式。FSMC 对存储器的访问分为异步模式和突发模式(同步模式)。在异步模式下，每次传送数据都需要产生一个确定的地址，而突发模式可以在开始时提供一个地址之后，把数据成组地连续写入。本实验使用异步模式 `FSMC_BurstAccessMode_Disable`。

FSMC_WaitSignalPolarity(配置等待信号极性)、**FSMC_WrapMode**(配置是否使用非对齐方式)、**FSMC_WaitSignalActive**(配置等待信号什么时期产生)、**FSMC_WaitSignal**(配置是否使用等待信号)、**FSMC_WriteBurst**(配置是否允许突发写操作)，这些成员均需要在突发模式开启后配置才有效。本实验使用的是异步模式，所以这些成员的参数没有意义。

FSMC_ReadWriteTimingStruct 和 **FSMC_WriteTimingStruct:** 用于设置读写时序。这两个变量都是 `FSMC_NORSRAMTimingInitTypeDef` 结构体指针类型。这两个参数在初始化的时候分别用来初始化片选控制寄存器 `FSMC_BTRx` 和写操

作时序控制寄存器 FSMC_BWTRx。FSMC_NORSRAMTimingInitTypeDef 结构体如下：

```
1.  typedef struct
2.  {
3.      uint32_t FSMC_AddressSetupTime; //地址建立时间
4.      uint32_t FSMC_AddressHoldTime; //地址保持时间
5.      uint32_t FSMC_DataSetupTime; //数据建立时间
6.      uint32_t FSMC_BusTurnAroundDuration; //总线恢复时间
7.      uint32_t FSMC_CLKDivision; //时钟分频
8.      uint32_t FSMC_DataLatency; //数据保持时间
9.      uint32_t FSMC_AccessMode; //访问模式
10. }FSMC_NORSRAMTimingInitTypeDef;
```

这些成员主要用于设计地址建立保持时间，数据建立时间等配置，这些时间是由 HCLK 经过成员时钟分频得来的，该分频值在成员 **FSMC_CLKDivision**(时钟分频)中设置，其中 **FSMC_AccessMode**(访问模式)成员的设置只在开启了扩展模式才有效，而且开启了扩展模式后，读时序和写时序的设置可以是独立的。本实验中我们需要读写速度不一样，所以开启了扩展模式并且对于参数 **FSMC_DataSetupTime** 设置了不同的值。此结构体其实就是对 FSMC_BTRx 和 FSMC_BWTRx 寄存器操作，大家可以查看中文参考手册寄存器说明。

本实验中的时序设置是根据 ILI9481 的数据手册设置的，调试的时候可以先把这些值设置得大一些，然后慢慢靠近数据手册要求的最小值，这样会取得比较好的效果。时序的参数设置对 LCD 的显示效果有一定的影响。

了解结构体成员功能后，就可以进行配置，本章实验配置代码如下：

```
1.  FSMC_NORSRAMInitTypeDef FSMC_NORSRAMInitStructure;
2.  FSMC_NORSRAMTimingInitTypeDef FSMC_ReadTimingInitStructure;
3.  FSMC_NORSRAMTimingInitTypeDef FSMC_WriteTimingInitStructure;
4.
5.  RCC_AHBPeriphClockCmd(RCC_AHBPeriph_FSMC, ENABLE); //使能 FSMC 时钟
6.
7.  FSMC_ReadTimingInitStructure.FSMC_AddressSetupTime = 0x01; //地址建立时间
    (ADDSET) 为 2 个 HCLK 1/36M=27ns
8.  FSMC_ReadTimingInitStructure.FSMC_AddressHoldTime = 0x00; //地址保持时间
    (ADDHLD) 模式 A 未用到
9.  FSMC_ReadTimingInitStructure.FSMC_DataSetupTime = 0x0f; // 数据保存时间
    为 16 个 HCLK,因为液晶驱动 IC 的读数据的时候，速度不能太快，尤其对 1289 这个 IC。
10. FSMC_ReadTimingInitStructure.FSMC_BusTurnAroundDuration = 0x00;
11. FSMC_ReadTimingInitStructure.FSMC_CLKDivision = 0x00;
12. FSMC_ReadTimingInitStructure.FSMC_DataLatency = 0x00;
```

```

13.  FSMC_ReadTimingInitStructure.FSMC_AccessMode = FSMC_AccessMode_A;    //模式
    A
14.
15.  FSMC_WriteTimingInitStructure.FSMC_AddressSetupTime = 0x15;    //地址建立时间
    (ADDSET) 为 16 个 HCLK
16.  FSMC_WriteTimingInitStructure.FSMC_AddressHoldTime = 0x15;    //地址保持时
    间
17.  FSMC_WriteTimingInitStructure.FSMC_DataSetupTime = 0x05;        //数据保存时
    间为 6 个 HCLK
18.  FSMC_WriteTimingInitStructure.FSMC_BusTurnAroundDuration = 0x00;
19.  FSMC_WriteTimingInitStructure.FSMC_CLKDivision = 0x00;
20.  FSMC_WriteTimingInitStructure.FSMC_DataLatency = 0x00;
21.  FSMC_WriteTimingInitStructure.FSMC_AccessMode = FSMC_AccessMode_A;    //模式
    A
22.
23.
24.  FSMC_NORSRAMInitStructure.FSMC_Bank = FSMC_Bank1_NORSRAM4; // 这里我们使用
    NE4 , 也就对应 BTCR[6],[7]。
25.  FSMC_NORSRAMInitStructure.FSMC_DataAddressMux = FSMC_DataAddressMux_Disable;
    // 不复用数据地址
26.  FSMC_NORSRAMInitStructure.FSMC_MemoryType =FSMC_MemoryType_SRAM; // FSMC_Memo
    ryType_SRAM; //SRAM
27.  FSMC_NORSRAMInitStructure.FSMC_MemoryDataWidth = FSMC_MemoryDataWidth_16b; //
    存储器数据宽度为 16bit
28.  FSMC_NORSRAMInitStructure.FSMC_BurstAccessMode =FSMC_BurstAccessMode_Disable;
    // FSMC_BurstAccessMode_Disable;
29.  FSMC_NORSRAMInitStructure.FSMC_WaitSignalPolarity = FSMC_WaitSignalPolarity_
    Low;
30.  FSMC_NORSRAMInitStructure.FSMC_AsynchronousWait=FSMC_AsynchronousWait_Disabl
    e;
31.  FSMC_NORSRAMInitStructure.FSMC_WrapMode = FSMC_WrapMode_Disable;
32.  FSMC_NORSRAMInitStructure.FSMC_WaitSignalActive = FSMC_WaitSignalActive_Befo
    reWaitState;
33.  FSMC_NORSRAMInitStructure.FSMC_WriteOperation = FSMC_WriteOperation_Enable;
    // 存储器写使能
34.  FSMC_NORSRAMInitStructure.FSMC_WaitSignal = FSMC_WaitSignal_Disable;
35.  FSMC_NORSRAMInitStructure.FSMC_ExtendedMode = FSMC_ExtendedMode_Enable; //
    读写使用不同的时序
36.  FSMC_NORSRAMInitStructure.FSMC_WriteBurst = FSMC_WriteBurst_Disable;
37.  FSMC_NORSRAMInitStructure.FSMC_ReadWriteTimingStruct = &FSMC_ReadTimingInitS
    tructure; //读写时序
38.  FSMC_NORSRAMInitStructure.FSMC_WriteTimingStruct = &FSMC_WriteTimingInitStru
    cture; //写时序
39.

```

40. `FSMC_NORSRAMInit(&FSMC_NORSRAMInitStructure);` //初始化 FSMC 配置

(3) 使能（开启）FSMC

固件库提供了不同的库函数来初始化各种存储器，同样也提供了不同类型的存储器使能函数，如下：

```
void FSMC_NORSRAMCmd(uint32_t FSMC_Bank, FunctionalState NewState);  
void FSMC_NANDCmd(uint32_t FSMC_Bank, FunctionalState NewState);  
void FSMC_PCCARDCmd(FunctionalState NewState);
```

这 3 个函数支持不同种类的存储器，从函数名来看也非常好理解。我们把 TFTLCD 当作 SRAM 使用，即使用第一个函数。该函数第一个参数用来选择存储器的区域，第二个参数用来使能或者失能。

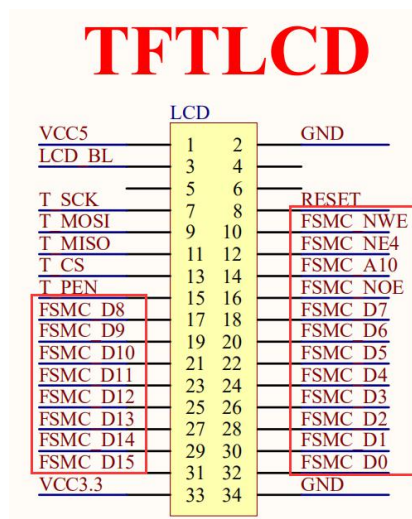
将以上几步全部配置好后，我们就可以使用 STM32F1 的 FSMC 了。

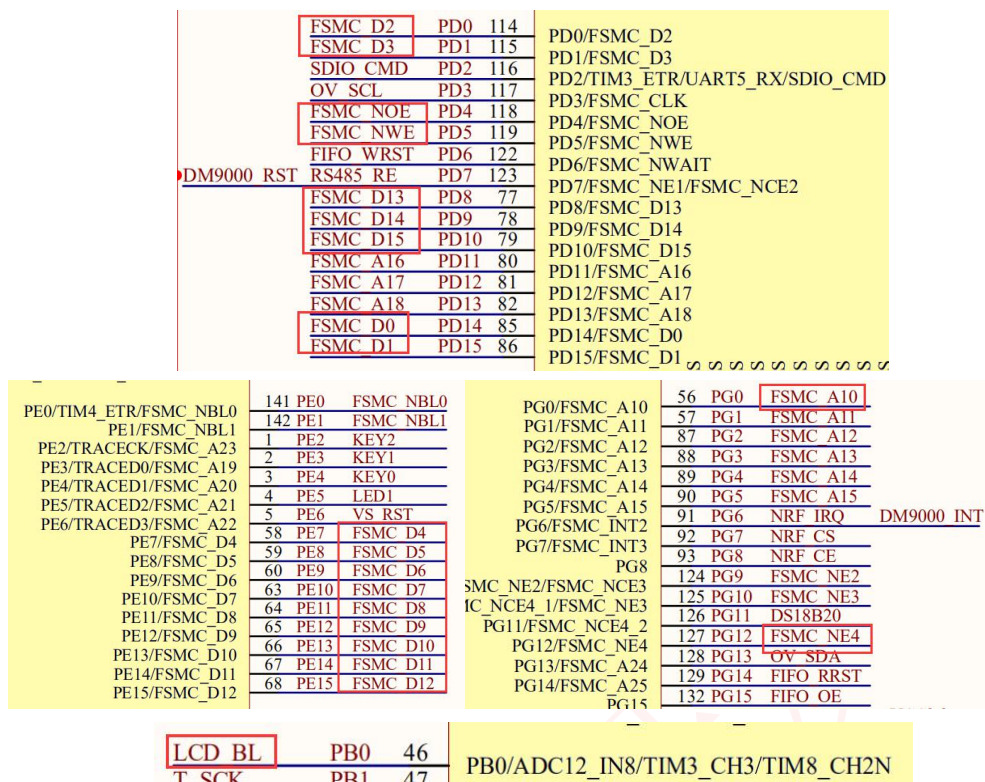
2.3 硬件设计

本实验使用到硬件资源如下：

- (1) DS0 指示灯
- (2) 串口 1
- (3) FSMC
- (4) TFTLCD 模块

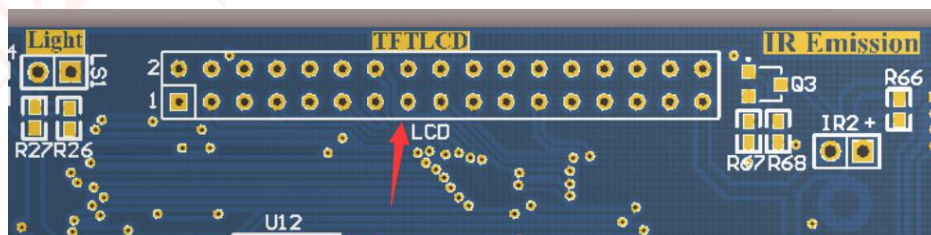
DS0 指示灯、串口 1 电路在前面章节都介绍过，这里就不多说，至于 FSMC 它属于 STM32F1 芯片内部的资源，只要通过软件配置好即可使用。下面我们看看 STM32F1 与 TFTLCD 接口的连接关系如图所示：





从电路图中可以看到，TFTLCD 接口 LCD1 连接在 STM32F1 的 FSMC 功能引脚上，TFTLCD 的数据口对应 FSMC_D0–FSMC_D15，TFTLCD 的 CS 对应 PG12 即 FMC_NE4，TFTLCD 的 RS 对应 PG0 即 FMC_A10，TFTLCD 的 WR 对应 PD5 即 FMC_NWE，TFTLCD 的 RD 对应 PD4 即 FMC_NOE，TFTLCD 的 LCD_BL 背光控制引脚对应 PB0（目前彩屏没有引出背光控制脚，预留）。TFTLCD 接口上的 T_SCK、T_MOSI 等引脚是用于控制触摸的，这些在后面触摸实验章节会介绍到。

因此我们只需要将 TFTLCD 模块插入开发板上 TFTLCD 接口即可，开发板上 TFTLCD 接口如下图所示：



DS0 指示灯用来提示系统运行状态，TFTLCD 的 ID 可以通过串口 1 打印输出。

2.4 软件设计

本章所要实现的功能是：在 TFTLCD 上显示 ASCII 字符、汉字和图片，同时

DS0 指示灯闪烁，提示系统正常运行。本实验我们使用的是 FSMC 的 Bank1 的第 4 区来控制 TFTLCD，程序框架如下：

- (1) 初始化 TFTLCD 对应的 GPIO，初始化 FSMC
- (2) TFTLCD 初始化，包括初始化序列
- (3) 编写 TFTLCD 的显示函数
- (4) 编写主函数

前面我们介绍了常用的 TFTLCD 初始化步骤，FSMC 配置步骤前面也已介绍，下面我们打开“\4--实验程序\1--基础实验\32-FSMC-TFTLCD 显示实验”工程，在 APP 工程组中可以看到添加了 tftlcd.c 文件（里面包含了多种 TFTLCD 驱动程序），在 StdPeriph_Driver 工程组中添加了 stm32f10x_fsmc.c 库文件。FSMC 操作的库函数都放在 stm32f10x_fsmc.c 和 stm32f10x_fsmc.h 文件中，所以使用到 FSMC 就必须加入 stm32f10x_fsmc.c 文件，同时还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

2.4.1 TFTLCD 的 GPIO 初始化函数

按照前面介绍的 TFTLCD 初始化流程，我们首先要初始化 TFTLCD 对应连接的 IO，其对应的 IO 前面硬件电路已介绍，代码如下：

```
1.  void TFTLCD_GPIO_Init(void)
2.  {
3.      GPIO_InitTypeDef  GPIO_InitStructure;
4.
5.      RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOD|RCC_APB
      2Periph_GPIOE|RCC_APB2Periph_GPIOG,ENABLE); //使能 PORTD,E,G 时钟
6.
7.      //PORTD 复用推挽输出
8.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1|GPIO_Pin_4|GPIO_Pin_
      5|GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_10|GPIO_Pin_14|GPIO_Pin_15; // //PORTD 复用推
      挽输出
9.      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
10.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
11.     GPIO_Init(GPIOD, &GPIO_InitStructure);
12.
13.     //PORTE 复用推挽输出
```

```

14.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7|GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_
10|GPIO_Pin_11|GPIO_Pin_12|GPIO_Pin_13|GPIO_Pin_14|GPIO_Pin_15; // //PORTD 复用
推挽输出
15.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
16.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
17.     GPIO_Init(GPIOE, &GPIO_InitStructure);

18.
19.     // //PORTG12 复用推挽输出 A10
20.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_12; // //PORTG 复用
推挽输出
21.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
22.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
23.     GPIO_Init(GPIOG, &GPIO_InitStructure);
24.
25.     //背光控制管脚初始化
26.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0; //PB0 推挽输出 背光
27.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
28.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
29.     GPIO_Init(GPIOB, &GPIO_InitStructure);
30.     LCD_LED=1; //点亮背光
31.
32. }

```

该函数非常简单，将对应的 IO 复用映射为 FSMC 功能，并配置 GPIO 为复用功能模式，最后调用 GPIO_Init 初始化。

2.4.2 TFTLCD 的 FSMC 初始化函数

本实验使用的是 FSMC 模拟 8080 时序，所以要对它配置，FSMC 初始化步骤前面已介绍，代码如下：

```

1.     void TFTLCD_FSMC_Init(void)
2.     {
3.         FSMC_NORSRAMInitTypeDef  FSMC_NORSRAMInitStructure;
4.         FSMC_NORSRAMTimingInitTypeDef  FSMC_ReadTimingInitStructure;
5.         FSMC_NORSRAMTimingInitTypeDef  FSMC_WriteTimingInitStructure;
6.
7.         RCC_AHBPeriphClockCmd(RCC_AHBPeriph_FSMC,ENABLE); //使能 FSMC 时钟
8.
9.         FSMC_ReadTimingInitStructure.FSMC_AddressSetupTime = 0x01; //地址建立时
间（ADDSET）为 2 个 HCLK 1/36M=27ns
10.        FSMC_ReadTimingInitStructure.FSMC_AddressHoldTime = 0x00; //地址保持时
间（ADDHLD）模式 A 未用到

```



```

11.     FSMC_ReadTimingInitStructure.FSMC_DataSetupTime = 0x0f;      // 数据保存
      时间为 16 个 HCLK, 因为液晶驱动 IC 的读数据的时候, 速度不能太快, 尤其对 1289 这个 IC。
12.     FSMC_ReadTimingInitStructure.FSMC_BusTurnAroundDuration = 0x00;
13.     FSMC_ReadTimingInitStructure.FSMC_CLKDivision = 0x00;
14.     FSMC_ReadTimingInitStructure.FSMC_DataLatency = 0x00;
15.     FSMC_ReadTimingInitStructure.FSMC_AccessMode = FSMC_AccessMode_A; // 模式
      A
16.
17.     FSMC_WriteTimingInitStructure.FSMC_AddressSetupTime = 0x15;  // 地址建立时
      间 (ADDSET) 为 16 个 HCLK
18.     FSMC_WriteTimingInitStructure.FSMC_AddressHoldTime = 0x15;  // 地址保持时
      间
19.     FSMC_WriteTimingInitStructure.FSMC_DataSetupTime = 0x05;      // 数据保
      存时间为 6 个 HCLK
20.     FSMC_WriteTimingInitStructure.FSMC_BusTurnAroundDuration = 0x00;
21.     FSMC_WriteTimingInitStructure.FSMC_CLKDivision = 0x00;
22.     FSMC_WriteTimingInitStructure.FSMC_DataLatency = 0x00;
23.     FSMC_WriteTimingInitStructure.FSMC_AccessMode = FSMC_AccessMode_A;  //
      模式 A
24.
25.
26.     FSMC_NORSRAMInitStructure.FSMC_Bank = FSMC_Bank1_NORSRAM4; // 这里我们使
      用 NE4 , 也就对应 BTCR[6],[7]。
27.     FSMC_NORSRAMInitStructure.FSMC_DataAddressMux = FSMC_DataAddressMux_Dis
      able; // 不复用数据地址
28.     FSMC_NORSRAMInitStructure.FSMC_MemoryType =FSMC_MemoryType_SRAM; // FSMC_
      MemoryType_SRAM;  //SRAM
29.     FSMC_NORSRAMInitStructure.FSMC_MemoryDataWidth = FSMC_MemoryDataWidth_16
      b; //存储器数据宽度为 16bit
30.     FSMC_NORSRAMInitStructure.FSMC_BurstAccessMode =FSMC_BurstAccessMode_Dis
      able; // FSMC_BurstAccessMode_Disable;
31.     FSMC_NORSRAMInitStructure.FSMC_WaitSignalPolarity = FSMC_WaitSignalPolar
      ity_Low;
32.     FSMC_NORSRAMInitStructure.FSMC_AsynchronousWait=FSMC_AsynchronousWait_Di
      sable;
33.     FSMC_NORSRAMInitStructure.FSMC_WrapMode = FSMC_WrapMode_Disable;
34.     FSMC_NORSRAMInitStructure.FSMC_WaitSignalActive = FSMC_WaitSignalActive_
      BeforeWaitState;
35.     FSMC_NORSRAMInitStructure.FSMC_WriteOperation = FSMC_WriteOperation_Enab
      le; // 存储器写使能
36.     FSMC_NORSRAMInitStructure.FSMC_WaitSignal = FSMC_WaitSignal_Disable;
37.     FSMC_NORSRAMInitStructure.FSMC_ExtendedMode = FSMC_ExtendedMode_Enable;
      // 读写使用不同的时序

```

```

38.     FSMC_NORSRAMInitStructure.FSMC_WriteBurst = FSMC_WriteBurst_Disable;
39.     FSMC_NORSRAMInitStructure.FSMC_ReadWriteTimingStruct = &FSMC_ReadTimingInitStructure; //读写时序
40.     FSMC_NORSRAMInitStructure.FSMC_WriteTimingStruct = &FSMC_WriteTimingInitStructure; //写时序
41.
42.     FSMC_NORSRAMInit(&FSMC_NORSRAMInitStructure); //初始化 FSMC 配置
43.
44.     FSMC_NORSRAMCmd(FSMC_Bank1_NORSRAM4, ENABLE); // 使能 BANK1
45.
46. }

```

该函数的功能很简单，按照前面讲解的 FSMC 结构体功能进行配置，由于 TFTLCD 的读写速度不一样，所以单独对 FSMC_NORSRAMTimingInitTypeDef 结构体进行读写时序的配置。可参考代码备注。

2.4.3 TFTLCD 初始化函数

要让 TFTLCD 显示，还需要初始化序列，即 TFT 彩屏厂家提供的 TFTLCD 寄存器设置值。将这些数值写入到 TFTLCD 内对应的命令寄存器中即可，所以还需要编写 TFTLCD 写命令和写数据等函数。TFTLCD 初始化代码如下：

```

1.  void TFTLCD_Init(void)
2.  {
3.      u16 i;
4.
5.      TFTLCD_GPIO_Init();
6.      TFTLCD_FSMC_Init();
7.
8.      delay_ms(50);
9.
10. #ifdef TFTLCD_HX8357D
11.     ... (省略)
12. #endif
13.
14. #ifdef TFTLCD_HX8357DN
15.     ... (省略)
16. #endif
17.
18. #ifdef TFTLCD_R61509V
19.     ... (省略)
20. #endif
21.

```

```
22.  #ifndef TFTLCD_R61509VN
23.      ... (省略)
24.  #endif
25.
26.  #ifndef TFTLCD_R61509V3
27.      ... (省略)
28.  #endif
29.
30.  #ifndef TFTLCD_HX8352C
31.      ... (省略)
32.  #endif
33.
34.  #ifndef TFTLCD_ILI9341
35.      ... (省略)
36.  #endif
37.
38.  #ifndef TFTLCD_ILI9327
39.      ... (省略)
40.  #endif
41.
42.  #ifndef TFTLCD_ILI9325
43.      ... (省略)
44.  #endif
45.
46.  #ifndef TFTLCD_ILI9486
47.      ... (省略)
48.  #endif
49.
50.  #ifndef TFTLCD_ST7793
51.      ... (省略)
52.  #endif
53.
54.  #ifndef TFTLCD_SSD1963
55.      ... (省略)
56.  #endif
57.
58.  #ifndef TFTLCD_NT35510
59.      ... (省略)
60.  #endif
61.
62.  #ifndef TFTLCD_ILI9481
63.      LCD_WriteCmd(0Xd3);
64.      tftlcd_data.id=TFTLCD->LCD_DATA;
65.      tftlcd_data.id=TFTLCD->LCD_DATA;
```

```
66.     tftlcd_data.id=TFTLCD->LCD_DATA;
67.     tftlcd_data.id<=8;
68.     tftlcd_data.id|=TFTLCD->LCD_DATA;
69. #endif
70.
71.     printf(" LCD ID:%x\r\n",tftlcd_data.id); //打印 LCD ID
72.
73. #ifdef TFTLCD_HX8357D
74.     ... (省略)
75. #endif
76.
77. #ifdef TFTLCD_HX8357DN
78.     ... (省略)
79. #endif
80.
81. #ifdef TFTLCD_R61509V
82.     ... (省略)
83. #endif
84.
85. #ifdef TFTLCD_R61509VN
86.     ... (省略)
87. #endif
88.
89. #ifdef TFTLCD_R61509V3
90.     ... (省略)
91. #endif
92.
93.
94. #ifdef TFTLCD_HX8352C
95.     ... (省略)
96. #endif
97.
98. #ifdef TFTLCD_ILI9341
99.     ... (省略)
100. #endif
101.
102. #ifdef TFTLCD_ILI9327
103.     ... (省略)
104. #endif
105.
106.
107. #ifdef TFTLCD_ILI9325
108.     ... (省略)
109. #endif
```

```
110.
111. #ifndef TFTLCD_ILI9486
112.     ... (省略)
113. #endif
114.
115. #ifndef TFTLCD_ST7793
116.     ... (省略)
117. #endif
118.
119. #ifndef TFTLCD_SSD1963
120.     ... (省略)
121. #endif
122.
123. #ifndef TFTLCD_NT35510
124.     ... (省略)
125. #endif
126.
127. #ifndef TFTLCD_ILI9481
128.     LCD_WriteCmd(0xFF);
129.     LCD_WriteCmd(0xFF);
130.     delay_ms(5);
131.
132.     LCD_WriteCmd(0xFF);
133.     LCD_WriteCmd(0xFF);
134.     LCD_WriteCmd(0xFF);
135.     LCD_WriteCmd(0xFF);
136.     delay_ms(10);
137.
138.     LCD_WriteCmd(0xB0);
139.     LCD_WriteData(0x00);
140.
141.     LCD_WriteCmd(0xB3);
142.     LCD_WriteData(0x02);
143.     LCD_WriteData(0x00);
144.     LCD_WriteData(0x00);
145.     LCD_WriteData(0x00);
146.
147.     LCD_WriteCmd(0xC0);
148.     LCD_WriteData(0x13);
149.     LCD_WriteData(0x3B); //480
150.     LCD_WriteData(0x00);
151.     LCD_WriteData(0x00);
152.     LCD_WriteData(0x00);
153.     LCD_WriteData(0x01);
```

```
154.    LCD_WriteData(0x00); //NW
155.    LCD_WriteData(0x43);
156.
157.    LCD_WriteCmd(0xC1);
158.    LCD_WriteData(0x08);
159.    LCD_WriteData(0x1B); //CLOCK
160.    LCD_WriteData(0x08);
161.    LCD_WriteData(0x08);
162.
163.    LCD_WriteCmd(0xC4);
164.    LCD_WriteData(0x11);
165.    LCD_WriteData(0x01);
166.    LCD_WriteData(0x73);
167.    LCD_WriteData(0x01);
168.
169.    LCD_WriteCmd(0xC6);
170.    LCD_WriteData(0x00);
171.
172.    LCD_WriteCmd(0xC8);
173.    LCD_WriteData(0x0F);
174.    LCD_WriteData(0x05);
175.    LCD_WriteData(0x14);
176.    LCD_WriteData(0x5C);
177.    LCD_WriteData(0x03);
178.    LCD_WriteData(0x07);
179.    LCD_WriteData(0x07);
180.    LCD_WriteData(0x10);
181.    LCD_WriteData(0x00);
182.    LCD_WriteData(0x23);
183.
184.    LCD_WriteData(0x10);
185.    LCD_WriteData(0x07);
186.    LCD_WriteData(0x07);
187.    LCD_WriteData(0x53);
188.    LCD_WriteData(0x0C);
189.    LCD_WriteData(0x14);
190.    LCD_WriteData(0x05);
191.    LCD_WriteData(0x0F);
192.    LCD_WriteData(0x23);
193.    LCD_WriteData(0x00);
194.
195.    LCD_WriteCmd(0x35);
196.    LCD_WriteData(0x00);
197.
```



```
198.    LCD_WriteCmd(0x44);
199.    LCD_WriteData(0x00);
200.    LCD_WriteData(0x01);
201.
202.    LCD_WriteCmd(0xD0);
203.    LCD_WriteData(0x07);
204.    LCD_WriteData(0x07); //VCI1
205.    LCD_WriteData(0x1D); //VRH
206.    LCD_WriteData(0x03); //BT
207.
208.    LCD_WriteCmd(0xD1);
209.    LCD_WriteData(0x03);
210.    LCD_WriteData(0x5B); //VCM
211.    LCD_WriteData(0x10); //VDV
212.
213.    LCD_WriteCmd(0xD2);
214.    LCD_WriteData(0x03);
215.    LCD_WriteData(0x24);
216.    LCD_WriteData(0x04);
217.
218.    LCD_WriteCmd(0x2A);
219.    LCD_WriteData(0x00);
220.    LCD_WriteData(0x00);
221.    LCD_WriteData(0x01);
222.    LCD_WriteData(0x3F); //320
223.
224.    LCD_WriteCmd(0x2B);
225.    LCD_WriteData(0x00);
226.    LCD_WriteData(0x00);
227.    LCD_WriteData(0x01);
228.    LCD_WriteData(0xDF); //480
229.
230.    LCD_WriteCmd(0x36);
231.    LCD_WriteData(0x00);
232.
233.    LCD_WriteCmd(0xC0);
234.    LCD_WriteData(0x13);
235.
236.    LCD_WriteCmd(0x3A);
237.    LCD_WriteData(0x55);
238.
239.    LCD_WriteCmd(0x11);
240.    delay_ms(150);
241.
```

```

242.     LCD_WriteCmd(0x29);
243.     delay_ms(30);
244.
245.     LCD_WriteCmd(0x2C);
246. #endif
247.
248.     LCD_Display_Dir(TFTLCD_DIR);           //0: 竖屏  1: 横屏  默认竖屏
249.     LCD_Clear(WHITE);
250. }

```

该函数中的 LCD_WriteCmd、LCD_WriteData 和 LCD_WriteData_Color 为写命令、写数据函数，其实现过程很简单，代码如下：

```

1.  //写寄存器函数
2.  //cmd:寄存器值
3.  void LCD_WriteCmd(u16 cmd)
4.  {
5.      #ifdef TFTLCD_HX8357D
6.          ... (省略)
7.      #endif
8.
9.      #ifdef TFTLCD_HX8357DN
10.         ... (省略)
11.      #endif
12.
13.     #ifdef TFTLCD_R61509V
14.         ... (省略)
15.     #endif
16.
17.     #ifdef TFTLCD_R61509VN
18.         ... (省略)
19.     #endif
20.
21.     #ifdef TFTLCD_R61509V3
22.         ... (省略)
23.     #endif
24.
25.     #ifdef TFTLCD_HX8352C
26.         ... (省略)
27.     #endif
28.
29.     #ifdef TFTLCD_ILI9341
30.         ... (省略)
31.     #endif
32.

```

```
33. #ifndef TFTLCD_ILI9327
34.     ... (省略)
35. #endif
36.
37. #ifndef TFTLCD_ILI9325
38.     ... (省略)
39. #endif
40.
41. #ifndef TFTLCD_ILI9486
42.     ... (省略)
43. #endif
44.
45. #ifndef TFTLCD_ST7793
46.     ... (省略)
47. #endif
48.
49. #ifndef TFTLCD_SSD1963
50.     ... (省略)
51. #endif
52.
53. #ifndef TFTLCD_NT35510
54.     ... (省略)
55. #endif
56.
57. #ifndef TFTLCD_ILI9481
58.     TFTLCD->LCD_CMD=cmd;
59. #endif
60.
61. }
62.
63. //写数据
64. //data:要写入的值
65. void LCD_WriteData(u16 data)
66. {
67.     #ifndef TFTLCD_HX8357D
68.         ... (省略)
69.     #endif
70.
71.     #ifndef TFTLCD_HX8357DN
72.         ... (省略)
73.     #endif
74.
75.     #ifndef TFTLCD_R61509V
76.         ... (省略)
```

```
77. #endif
78.
79. #ifdef TFTLCD_R61509VN
80.     ... (省略)
81. #endif
82.
83. #ifdef TFTLCD_R61509V3
84.     ... (省略)
85. #endif
86.
87. #ifdef TFTLCD_HX8352C
88.     ... (省略)
89. #endif
90.
91. #ifdef TFTLCD_ILI9341
92.     ... (省略)
93. #endif
94.
95. #ifdef TFTLCD_ILI9327
96.     ... (省略)
97. #endif
98.
99. #ifdef TFTLCD_ILI9325
100.    ... (省略)
101. #endif
102.
103. #ifdef TFTLCD_ILI9486
104.    ... (省略)
105. #endif
106.
107. #ifdef TFTLCD_ST7793
108.    ... (省略)
109. #endif
110.
111. #ifdef TFTLCD_SSD1963
112.    ... (省略)
113. #endif
114.
115. #ifdef TFTLCD_NT35510
116.    ... (省略)
117. #endif
118.
119. #ifdef TFTLCD_ILI9481
120.    TFTLCD->LCD_DATA=data;
```

```
121. #endif
122. }
123.
124. void LCD_WriteCmdData(u16 cmd,u16 data)
125. {
126.     LCD_WriteCmd(cmd);
127.     LCD_WriteData(data);
128. }
129.
130. void LCD_WriteData_Color(u16 color)
131. {
132. #ifdef TFTLCD_HX8357D
133.     ... (省略)
134. #endif
135.
136. #ifdef TFTLCD_HX8357DN
137.     ... (省略)
138. #endif
139.
140. #ifdef TFTLCD_R61509V
141.     ... (省略)
142. #endif
143.
144. #ifdef TFTLCD_R61509VN
145.     ... (省略)
146. #endif
147.
148. #ifdef TFTLCD_R61509V3
149.     ... (省略)
150. #endif
151.
152. #ifdef TFTLCD_HX8352C
153.     ... (省略)
154. #endif
155.
156. #ifdef TFTLCD_ILI9341
157.     ... (省略)
158. #endif
159.
160. #ifdef TFTLCD_ILI9327
161.     ... (省略)
162. #endif
163.
164. #ifdef TFTLCD_ILI9325
```

```

165.     ... (省略)
166. #endif
167.
168. #ifdef TFTLCD_ILI9486
169.     ... (省略)
170. #endif
171.
172. #ifdef TFTLCD_ST7793
173.     ... (省略)
174. #endif
175.
176. #ifdef TFTLCD_SSD1963
177.     ... (省略)
178. #endif
179.
180. #ifdef TFTLCD_NT35510
181.     ... (省略)
182. #endif
183.
184. #ifdef TFTLCD_ILI9481
185.     TFTLCD->LCD_DATA=color;
186. #endif
187. }

```

函数中的 TFTLCD 是一个结构体类型指针宏定义，因为我们使用的 FSMC 是 Bank1 的第 4 区，并且把 FSMC_A10 作为数据命令选择。具体的定义在 tftlcd.h 文件开始处，代码如下：

```

1.  //TFTLCD 地址结构体
2.  typedef struct
3.  {
4.      u16 LCD_CMD;
5.      u16 LCD_DATA;
6.  }TFTLCD_TypeDef;
7.
8.
9.  //使用 NOR/SRAM 的 Bank1.sector4,地址位 HADDR[27,26]=11 A10 作为数据命令区分线
10. //注意设置 16 位总线时 STM32 内部会右移一位对齐!
11. #define TFTLCD_BASE      ((u32)(0x6C000000 | 0x000007FE))
12. #define TFTLCD           ((TFTLCD_TypeDef *) TFTLCD_BASE)

```

其中 TFTLCD_BASE，必须根据我们外部电路的连接来确定，我们使用 Bank1.sector4 就是从地址 0X6C000000 开始，而 0X000007FE，则是 A10 的偏移量，很多朋友不理解这个偏移量的概念，我们简单说明下：以 A10 为例，7FE

转换成二进制就是： 111 1111 1110，而 16 位数据时，地址右移一位对齐（介绍框图时已介绍），那么实际对应到地址引脚的时候，就是： A10:A0=011 1111 1111，此时 A10 是 0，但是如果 16 位地址再加 1（注意：对应到 8 位地址是加 2，即 7FE+0X02），那么： A10:A0=100 0000 0000，此时 A10 就是 1 了，即实现了对 RS 的 0 和 1 的控制。

我们将这个地址强制转换为 TFTLCD_TypeDef 结构体地址，那么可以得到 TFTLCD->LCD_CMD 的地址就是 0X6C00,07FE，对应 A10 的状态为 0(即 RS=0)，而 TFTLCD->LCD_DATA 的地址就是 0X6C00,0800（结构体地址自增），对应 A10 的状态为 1（即 RS=1）。

所以，有了这个定义，当我们要往 LCD 写命令/数据的时候，可以这样写：

```
TFTLCD->LCD_CMD=cmd;//写命令
TFTLCD->LCD_DATA=data;//写数据
```

而读的时候反过来操作就可以了，如下所示：

```
cmd=TFTLCD->LCD_CMD;
data=TFTLCD->LCD_DATA;
```

这其中，CS、WR、RD 和 IO 口方向都是由 FSMC 控制，不需要我们手动设置。下面我们再来介绍下 tftlcd.h 中另一个重要的结构体：

```
//LCD 重要参数集
typedef struct
{
    u16 width;          //LCD 宽度
    u16 height;         //LCD 高度
    u16 id;             //LCD ID
    u8  dir;            //横屏还是竖屏控制：0，竖屏；1，横屏。
} _tftlcd_data;
extern _tftlcd_data tftlcd_data; //管理 TFTLCD 重要参数
```

该结构体用于保存一些 TFTLCD 重要参数信息，比如 LCD 的尺寸、LCD ID（驱动 IC 型号）、LCD 显示方向，当然你也可以在我们这个结构体内添加其他 LCD 相关参数变量。通过对此结构体的管理可以让我们的驱动函数支持不同尺寸的 LCD，同时可以实现 LCD 横竖屏切换等重要功能。所以大家今后要多学习使用结构体管理不同属性。

初始化函数内我们还使用了条件编译语句来兼容多种彩屏驱动程序。比如：

```
#ifdef TFTLCD_HX8357D
....对应 HX8357D 驱动的操作代码
#endif
```



```

#ifdef TFTLCD_HX8352C
.... 对应 HX8352C 驱动的操作代码
#endif
#ifdef TFTLCD_NT5510
.... 对应 NT5510 驱动的操作代码
#endif
#ifdef TFTLCD_ILI9481
.... 对应 ILI9481 驱动的操作代码
#endif

```

本章所介绍的 TFTLCD 驱动 IC 是 ILI9481，所以要对这部分代码操作，只需在 tftlcd.h 文件开始处开启这个宏定义，其他彩屏的宏定义就关闭它。如果您手上使用的是其他驱动类型屏，对应型号打开即可，如下：（后面如果又增加了屏，使用同样方法来兼容）

```

1.  //定义 LCD 彩屏的驱动类型 可根据自己手上的彩屏背面型号来选择打开哪种驱动
2.  // #define TFTLCD_HX8357D
3.  // #define TFTLCD_HX8352C
4.  // #define TFTLCD_ILI9341
5.  // #define TFTLCD_ILI9327
6.  // #define TFTLCD_ILI9486
7.  // #define TFTLCD_R61509V
8.  // #define TFTLCD_R61509VN
9.  // #define TFTLCD_R61509V3
10. // #define TFTLCD_ST7793
11. // #define TFTLCD_NT35510
12. // #define TFTLCD_HX8357DN
13. // #define TFTLCD_ILI9325
14. // #define TFTLCD_SSD1963
15. #define TFTLCD_ILI9481

```

在初始化函数最后部分还调用了 LCD_Display_Dir 和 LCD_Clear 函数，LCD_Display_Dir 函数用于切换 TFTLCD 显示方向（横屏和竖屏），默认我们设置为竖屏，代码如下：

```

1.  //设置 LCD 显示方向
2.  //dir:0, 竖屏; 1, 横屏
3.  void LCD_Display_Dir(u8 dir)
4.  {
5.      tftlcd_data.dir=dir;          //横屏/竖屏
6.      if(dir==0) //默认竖屏方向
7.      {
8.          #ifdef TFTLCD_HX8357D
9.              ... (省略)

```

```
10. #endif
11.
12. #ifdef TFTLCD_HX8357DN
13.     ... (省略)
14. #endif
15.
16. #ifdef TFTLCD_R61509V
17.     ... (省略)
18. #endif
19.
20. #ifdef TFTLCD_R61509VN
21.     ... (省略)
22. #endif
23.
24. #ifdef TFTLCD_R61509V3
25.     ... (省略)
26. #endif
27.
28. #ifdef TFTLCD_HX8352C
29.     ... (省略)
30. #endif
31.
32. #ifdef TFTLCD_ILI9341
33.     ... (省略)
34. #endif
35.
36. #ifdef TFTLCD_ILI9327
37.     ... (省略)
38. #endif
39.
40. #ifdef TFTLCD_ILI9325
41.     ... (省略)
42. #endif
43.
44. #ifdef TFTLCD_ILI9486
45.     ... (省略)
46. #endif
47.
48. #ifdef TFTLCD_ST7793
49.     ... (省略)
50. #endif
51.
52. #ifdef TFTLCD_SSD1963
53.     ... (省略)
```

```
54. #endif
55.
56. #ifdef TFTLCD_NT35510
57.     ... (省略)
58. #endif
59.
60. #ifdef TFTLCD_ILI9481
61.     LCD_WriteCmd(0x36);    //设置彩屏显示方向的寄存器
62.     LCD_WriteData(0x00);
63.     tftlcd_data.height=480;
64.     tftlcd_data.width=320;
65. #endif
66. }
67. else
68. {
69. #ifdef TFTLCD_HX8357D
70.     ... (省略)
71. #endif
72.
73. #ifdef TFTLCD_HX8357DN
74.     ... (省略)
75. #endif
76.
77. #ifdef TFTLCD_R61509V
78.     ... (省略)
79. #endif
80.
81. #ifdef TFTLCD_R61509VN
82.     ... (省略)
83. #endif
84.
85. #ifdef TFTLCD_R61509V3
86.     ... (省略)
87. #endif
88.
89. #ifdef TFTLCD_HX8352C
90.     ... (省略)
91. #endif
92.
93. #ifdef TFTLCD_ILI9341
94.     ... (省略)
95. #endif
96.
97. #ifdef TFTLCD_ILI9327
```

```

98.         ... (省略)
99.     #endif
100.
101. #ifdef TFTLCD_ILI9325
102.     ... (省略)
103. #endif
104.
105. #ifdef TFTLCD_ILI9486
106.     ... (省略)
107. #endif
108.
109. #ifdef TFTLCD_ST7793
110.     ... (省略)
111. #endif
112.
113. #ifdef TFTLCD_SSD1963
114.     ... (省略)
115. #endif
116.
117. #ifdef TFTLCD_NT35510
118.     ... (省略)
119. #endif
120.
121. #ifdef TFTLCD_ILI9481
122.     LCD_WriteCmd(0x36);    //设置彩屏显示方向的寄存器
123.     LCD_WriteData(0x60);
124.     tftlcd_data.height=320;
125.     tftlcd_data.width=480;
126. #endif
127. }
128. }

```

横屏和竖屏的控制只需要对 TFTLCD 显示方向寄存器进行操作（寄存器可通过彩屏数据手册查找），然后再把他们的 X 和 Y 轴像素调换即可。

为了方便大家设置横竖屏，我们直接在初始化函数内调用 `LCD_Display_Dir()` 函数时传递的是一个宏定义 `TFTLCD_DIR`，而该宏定义在 `tftlcd.h` 头文件开始处也定义了，其实就是数字 0 或者 1，如果要横屏显示的话该宏值为 1，如果要竖屏显示就为 0。如下所示：

```
#define TFTLCD_DIR 0 //0: 竖屏 1: 横屏 默认竖屏
```

`LCD_Clear` 函数用于清屏，函数入口参数用于选择清屏的颜色，常用的颜色都在 `tftlcd.h` 文件内进行了宏定义，直接调用宏即可。清屏函数实现过程很简

单，其实就是设置 TFTLCD 显示窗口，然后写入

tftlcd_data.width*tftlcd_data.height 个颜色数据即可。设置窗口函数代码如下：

```
1. //设置窗口,并自动设置画点坐标到窗口左上角(sx,sy).
2. //sx,sy:窗口起始坐标(左上角)
3. //width,height:窗口宽度和高度,必须大于0!!
4. //窗体大小:width*height.
5. void LCD_Set_Window(u16 sx,u16 sy,u16 width,u16 height)
6. {
7.     #ifdef TFTLCD_HX8357D
8.         ... (省略)
9.     #endif
10.
11.     #ifdef TFTLCD_HX8357DN
12.         ... (省略)
13.     #endif
14.
15.     #ifdef TFTLCD_R61509V
16.         ... (省略)
17.     #endif
18.
19.     #ifdef TFTLCD_R61509VN
20.         ... (省略)
21.     #endif
22.
23.     #ifdef TFTLCD_R61509V3
24.         ... (省略)
25.     #endif
26.
27.     #ifdef TFTLCD_HX8352C
28.         ... (省略)
29.     #endif
30.
31.     #ifdef TFTLCD_ILI9341
32.         ... (省略)
33.     #endif
34.
35.     #ifdef TFTLCD_ILI9327
36.         ... (省略)
37.     #endif
38.
39.     #ifdef TFTLCD_ILI9325
40.         ... (省略)
```

```

41. #endif
42.
43. #ifdef TFTLCD_ILI9486
44.     ... (省略)
45. #endif
46.
47. #ifdef TFTLCD_ST7793
48.     ... (省略)
49. #endif
50.
51. #ifdef TFTLCD_SSD1963
52.     ... (省略)
53. #endif
54.
55. #ifdef TFTLCD_NT35510
56.     ... (省略)
57. #endif
58.
59. #ifdef TFTLCD_ILI9481
60.     LCD_WriteCmd(0x2A);
61.     LCD_WriteData(sx/256);
62.     LCD_WriteData(sx%256);
63.     LCD_WriteData(width/256);
64.     LCD_WriteData(width%256);
65.
66.     LCD_WriteCmd(0x2B);
67.     LCD_WriteData(sy/256);
68.     LCD_WriteData(sy%256);
69.     LCD_WriteData(height/256);
70.     LCD_WriteData(height%256);
71.
72.     LCD_WriteCmd(0x2C);
73. #endif
74. }

```

其实就是在 TFTLCD 的 X 方向和 Y 方向寄存器内写入窗口值。我们讲解的 ILI9481 的 X 方向和 Y 方向命令（寄存器）是 0x2A 和 0x2B，写完窗口值还需要将这些值写入到 TFTLCD 的 GRAM 内，命令是 0x2C。这些命令也是通过彩屏数据手册查找。

在初始化函数内使用了 printf 函数打印 TFTLCD 的 ID，所以在主函数内需要对串口初始化，否则将导致程序死在 printf 里面，如果不想用 printf，那么请注释掉它。

2.4.4 TFTLCD 显示函数

上述几个函数完成后,我们就可以使用 TFTLCD 了,要在 TFTLCD 上显示内容,我们需要编写对应的显示函数,比如要显示一个点,那么就编写一个点的函数。

TFTLCD 显示函数如下:

```
1.  //清屏函数
2.  //color:要清屏的填充色
3.  void LCD_Clear(u16 color)
4.  {
5.      uint16_t i, j ;
6.
7.      LCD_Set_Window(0, 0, tftlcd_data.width-1, tftlcd_data.height-1);    //
    作用区域
8.      for(i=0; i<tftlcd_data.width; i++)
9.      {
10.         for (j=0; j<tftlcd_data.height; j++)
11.         {
12.             LCD_WriteData_Color(color);
13.         }
14.     }
15. }
16.
17.
18. //在指定区域内填充单个颜色
19. // (sx,sy),(ex,ey):填充矩形对角坐标,区域大小为:(ex-sx+1)*(ey-sy+1)
20. //color:要填充的颜色
21. void LCD_Fill(u16 xState,u16 yState,u16 xEnd,u16 yEnd,u16 color)
22. {
23.     uint16_t temp;
24.
25.     if((xState > xEnd) || (yState > yEnd))
26.     {
27.         return;
28.     }
29.     LCD_Set_Window(xState, yState, xEnd, yEnd);
30.     xState = xEnd - xState + 1;
31.     yState = yEnd - yState + 1;
32.
33.     while(xState--)
34.     {
35.         temp = yState;
36.         while (temp--)
```



```

37.     {
38.         LCD_WriteData_Color(color);
39.     }
40. }
41. }
42.
43. //在指定区域内填充指定颜色块
44. //(sx,sy),(ex,ey):填充矩形对角坐标,区域大小为:(ex-sx+1)*(ey-sy+1)
45. //color:要填充的颜色
46. void LCD_Color_Fill(u16 sx,u16 sy,u16 ex,u16 ey,u16 *color)
47. {
48.     u16 height,width;
49.     u16 i,j;
50.     width=ex-sx+1;        //得到填充的宽度
51.     height=ey-sy+1;       //高度
52.
53.     LCD_Set_Window(sx, sy,ex, ey);
54.     for(i=0;i<height;i++)
55.     {
56.         for(j=0;j<width;j++)
57.         {
58.             LCD_WriteData_Color(color[i*width+j]);
59.         }
60.     }
61. }
62. //画点
63. //x,y:坐标
64. //FRONT_COLOR:此点的颜色
65. void LCD_DrawPoint(u16 x,u16 y)
66. {
67.     LCD_Set_Window(x, y, x, y); //设置点的位置
68.     LCD_WriteData_Color(FRONT_COLOR);
69. }
70.
71. //快速画点
72. //x,y:坐标
73. //color:颜色
74. void LCD_DrawFRONT_COLOR(u16 x,u16 y,u16 color)
75. {
76.     LCD_Set_Window(x, y, x, y);
77.     LCD_WriteData_Color(color);
78. }
79.
80. //画线

```

```

81. //x1,y1:起点坐标
82. //x2,y2:终点坐标
83. void LCD_DrawLine(u16 x1, u16 y1, u16 x2, u16 y2)
84. {
85.     u16 t;
86.     int xerr=0,yerr=0,delta_x,delta_y,distance;
87.     int incx,incy,uRow,uCol;
88.     delta_x=x2-x1; //计算坐标增量
89.     delta_y=y2-y1;
90.     uRow=x1;
91.     uCol=y1;
92.     if(delta_x>0)incx=1; //设置单步方向
93.     else if(delta_x==0)incx=0; //垂直线
94.     else {incx=-1;delta_x=-delta_x;}
95.     if(delta_y>0)incy=1;
96.     else if(delta_y==0)incy=0; //水平线
97.     else{incy=-1;delta_y=-delta_y;}
98.     if( delta_x>delta_y)distance=delta_x; //选取基本增量坐标轴
99.     else distance=delta_y;
100.    for(t=0;t<=distance+1;t++ )//画线输出
101.    {
102.        LCD_DrawPoint(uRow,uCol); //画点
103.        xerr+=delta_x ;
104.        yerr+=delta_y ;
105.        if(xerr>distance)
106.        {
107.            xerr-=distance;
108.            uRow+=incx;
109.        }
110.        if(yerr>distance)
111.        {
112.            yerr-=distance;
113.            uCol+=incy;
114.        }
115.    }
116. }
117.
118. void LCD_DrawLine_Color(u16 x1, u16 y1, u16 x2, u16 y2,u16 color)
119. {
120.     u16 t;
121.     int xerr=0,yerr=0,delta_x,delta_y,distance;
122.     int incx,incy,uRow,uCol;
123.     delta_x=x2-x1; //计算坐标增量
124.     delta_y=y2-y1;

```

```

125.     uRow=x1;
126.     uCol=y1;
127.     if(delta_x>0)incx=1; //设置单步方向
128.     else if(delta_x==0)incx=0; //垂直线
129.     else {incx=-1;delta_x=-delta_x;}
130.     if(delta_y>0)incy=1;
131.     else if(delta_y==0)incy=0; //水平线
132.     else{incy=-1;delta_y=-delta_y;}
133.     if( delta_x>delta_y)distance=delta_x; //选取基本增量坐标轴
134.     else distance=delta_y;
135.     for(t=0;t<=distance+1;t++ )//画线输出
136.     {
137.         LCD_DrawFRONT_COLOR(uRow,uCol,color); //画点
138.         xerr+=delta_x ;
139.         yerr+=delta_y ;
140.         if(xerr>distance)
141.         {
142.             xerr-=distance;
143.             uRow+=incx;
144.         }
145.         if(yerr>distance)
146.         {
147.             yerr-=distance;
148.             uCol+=incy;
149.         }
150.     }
151. }
152.
153.
154. // 画一个十字的标记
155. // x: 标记的 X 坐标
156. // y: 标记的 Y 坐标
157. // color: 标记的颜色
158. void LCD_DrowSign(uint16_t x, uint16_t y, uint16_t color)
159. {
160.     uint8_t i;
161.
162.     /* 画点 */
163.     LCD_Set_Window(x-1, y-1, x+1, y+1);
164.     for(i=0; i<9; i++)
165.     {
166.         LCD_WriteData_Color(color);
167.     }
168.

```

```

169.     /* 画竖 */
170.     LCD_Set_Window(x-4, y, x+4, y);
171.     for(i=0; i<9; i++)
172.     {
173.         LCD_WriteData_Color(color);
174.     }
175.
176.     /* 画横 */
177.     LCD_Set_Window(x, y-4, x, y+4);
178.     for(i=0; i<9; i++)
179.     {
180.         LCD_WriteData_Color(color);
181.     }
182. }
183.
184. //画矩形
185. //(x1,y1),(x2,y2):矩形的对角坐标
186. void LCD_DrawRectangle(u16 x1, u16 y1, u16 x2, u16 y2)
187. {
188.     LCD_DrawLine(x1,y1,x2,y1);
189.     LCD_DrawLine(x1,y1,x1,y2);
190.     LCD_DrawLine(x1,y2,x2,y2);
191.     LCD_DrawLine(x2,y1,x2,y2);
192. }
193. //在指定位置画一个指定大小的圆
194. //(x,y):中心点
195. //r      :半径
196. void LCD_Draw_Circle(u16 x0,u16 y0,u8 r)
197. {
198.     int a,b;
199.     int di;
200.     a=0;b=r;
201.     di=3-(r<<1);           //判断下个点位置的标志
202.     while(a<=b)
203.     {
204.         LCD_DrawPoint(x0+a,y0-b);           //5
205.         LCD_DrawPoint(x0+b,y0-a);           //0
206.         LCD_DrawPoint(x0+b,y0+a);           //4
207.         LCD_DrawPoint(x0+a,y0+b);           //6
208.         LCD_DrawPoint(x0-a,y0+b);           //1
209.         LCD_DrawPoint(x0-b,y0+a);
210.         LCD_DrawPoint(x0-a,y0-b);           //2
211.         LCD_DrawPoint(x0-b,y0-a);           //7
212.         a++;

```

```

213.         //使用 Bresenham 算法画圆
214.         if(di<0)di +=4*a+6;
215.         else
216.         {
217.             di+=10+4*(a-b);
218.             b--;
219.         }
220.     }
221. }
222.
223.
224.
225. //在指定位置显示一个字符
226. //x,y:起始坐标
227. //num:要显示的字符:" --->"~"
228. //size:字体大小 12/16/24
229. //mode:叠加方式(1)还是非叠加方式(0)
230. void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode)
231. {
232.     u8 temp,t1,t;
233.     u16 y0=y;
234.     u8 csize=(size/8+((size%8)?1:0))*(size/2);    //得到字体一个字符对应点阵
        集所占的字节数
235.     num=num-' ';//得到偏移后的值 (ASCII 字库是从空格开始取模, 所以-' '就是对应字符
        的字库)
236.     for(t=0;t<csize;t++)
237.     {
238.         if(size==12)temp=ascii_1206[num][t];    //调用 1206 字体
239.         else if(size==16)temp=ascii_1608[num][t];    //调用 1608 字体
240.         else if(size==24)temp=ascii_2412[num][t];    //调用 2412 字体
241.         else return;    //没有的字库
242.         for(t1=0;t1<8;t1++)
243.         {
244.             if(temp&0x80)LCD_DrawFRONT_COLOR(x,y,FRONT_COLOR);
245.             else if(mode==0)LCD_DrawFRONT_COLOR(x,y,BACK_COLOR);
246.             temp<<=1;
247.             y++;
248.             if(y>=tftlcd_data.height)return;    //超区域了
249.             if((y-y0)==size)
250.             {
251.                 y=y0;
252.                 x++;
253.                 if(x>=tftlcd_data.width)return;    //超区域了
254.                 break;

```

```

255.         }
256.     }
257. }
258. }
259. //m^n 函数
260. //返回值:m^n 次方.
261. u32 LCD_Pow(u8 m,u8 n)
262. {
263.     u32 result=1;
264.     while(n-->0)result*=m;
265.     return result;
266. }
267. //显示数字,高位为 0,则不显示
268. //x,y :起点坐标
269. //len :数字的位数
270. //size:字体大小
271. //color:颜色
272. //num:数值(0~4294967295);
273. void LCD_ShowNum(u16 x,u16 y,u32 num,u8 len,u8 size)
274. {
275.     u8 t,temp;
276.     u8 enshow=0;
277.     for(t=0;t<len;t++)
278.     {
279.         temp=(num/LCD_Pow(10,len-t-1))%10;
280.         if(enshow==0&&t<(len-1))
281.         {
282.             if(temp==0)
283.             {
284.                 LCD_ShowChar(x+(size/2)*t,y,' ',size,0);
285.                 continue;
286.             }else enshow=1;
287.         }
288.         LCD_ShowChar(x+(size/2)*t,y,temp+'0',size,0);
289.     }
290. }
291. }
292.
293. //显示数字,高位为 0,还是显示
294. //x,y:起点坐标
295. //num:数值(0~999999999);
296. //len:长度(即要显示的位数)
297. //size:字体大小
298. //mode:

```

```

299. // [7]:0, 不填充;1, 填充 0.
300. // [6:1]: 保留
301. // [0]:0, 非叠加显示;1, 叠加显示.
302. void LCD_ShowxNum(u16 x,u16 y,u32 num,u8 len,u8 size,u8 mode)
303. {
304.     u8 t,temp;
305.     u8 enshow=0;
306.     for(t=0;t<len;t++)
307.     {
308.         temp=(num/LCD_Pow(10,len-t-1))%10;
309.         if(enshow==0&&t<(len-1))
310.         {
311.             if(temp==0)
312.             {
313.                 if(mode&0X80)LCD_ShowChar(x+(size/2)*t,y,'0',size,mode&0X01);
314.
315.                 else LCD_ShowChar(x+(size/2)*t,y,' ',size,mode&0X01);
316.                 continue;
317.             }else enshow=1;
318.         }
319.         LCD_ShowChar(x+(size/2)*t,y,temp+'0',size,mode&0X01);
320.     }
321. }
322. //显示字符串
323. //x,y:起点坐标
324. //width,height:区域大小
325. //size:字体大小
326. //*p:字符串起始地址
327. void LCD_ShowString(u16 x,u16 y,u16 width,u16 height,u8 size,u8 *p)
328. {
329.     u8 x0=x;
330.     width+=x;
331.     height+=y;
332.     while((*p<='~')&&(*p>=' '))//判断是不是非法字符!
333.     {
334.         if(x>=width){x=x0;y+=size;}
335.         if(y>=height)break;//退出
336.         LCD_ShowChar(x,y,*p,size,0);
337.         x+=size/2;
338.         p++;
339.     }
340. }
341.

```

```

342. /*****
    *
343. *函数名: LCD_ShowFontHZ
344. *输 入: x: 汉字显示的 X 坐标
345. *      * y: 汉字显示的 Y 坐标
346. *      * cn: 要显示的汉字
347. *      * wordColor: 文字的颜色
348. *      * backColor: 背景颜色
349. *输 出:
350. *功 能: 写二号楷体汉字
351. *****/
    /
352. #if 1
353. void LCD_ShowFontHZ(u16 x, u16 y, u8 *cn)
354. {
355.     u8 i, j, wordNum;
356.     u16 color;
357.     u16 x0=x;
358.     u16 y0=y;
359.     while (*cn != '\0')
360.     {
361.         for (wordNum=0; wordNum<20; wordNum++)
362.         { //wordNum 扫描字库的字数
363.             if ((CnChar32x29[wordNum].Index[0]==*cn)
364.                 &&(CnChar32x29[wordNum].Index[1]==*(cn+1)))
365.             {
366.                 for(i=0; i<116; i++)
367.                 { //MSK 的位数
368.                     color=CnChar32x29[wordNum].Msk[i];
369.                     for(j=0; j<8; j++)
370.                     {
371.                         if((color&0x80)==0x80)
372.                         {
373.                             LCD_DrawFRONT_COLOR(x,y,FRONT_COLOR);
374.                         }
375.                         else
376.                         {
377.                             LCD_DrawFRONT_COLOR(x,y,BACK_COLOR);
378.                         }
379.                         color<<=1;
380.                         x++;
381.                         if((x-x0)==32)
382.                         {
383.                             x=x0;

```



```

384.             y++;
385.             if((y-y0)==29)
386.             {
387.                 y=y0;
388.             }
389.         }
390.     } //for(j=0;j<8;j++)结束
391. }
392. }
393.
394. } //for (wordNum=0; wordNum<20; wordNum++)结束
395.     cn += 2;
396.     x += 32;
397.     x0=x;
398. }
399. }
400. #endif
401.
402. void LCD_ShowPicture(u16 x, u16 y, u16 wide, u16 high,u8 *pic)
403. {
404.     u16 temp = 0;
405.     long tmp=0,num=0;
406.     LCD_Set_Window(x, y, x+wide-1, y+high-1);
407.     num = wide * high*2 ;
408.     do
409.     {
410.         temp = pic[tmp + 1];
411.         temp = temp << 8;
412.         temp = temp | pic[tmp];
413.         LCD_WriteData_Color(temp); //逐点显示
414.         tmp += 2;
415.     }
416.     while(tmp < num);
417. }

```

我们给大家提供很多 TFTLCD 操作的 API 函数，如下：

1. void TFTLCD_Init(void); //初始化
2. void LCD_Set_Window(u16 sx,u16 sy,u16 width,u16 height); //设置窗口
3. void LCD_Display_Dir(u8 dir); //设置屏幕显示方向
4. void LCD_Clear(u16 Color); //清屏
5. void LCD_Fill(u16 xState,u16 yState,u16 xEnd,u16 yEnd,u16 color); //填充单色
6. void LCD_Color_Fill(u16 sx,u16 sy,u16 ex,u16 ey,u16 *color); //在指定区域内填充指定颜色块
7. void LCD_DrawPoint(u16 x,u16 y); //画点
8. void LCD_DrawFRONT_COLOR(u16 x,u16 y,u16 color); //指定颜色画点

```

9.    u16 LCD_ReadPoint(u16 x,u16 y);//读点
10.   void LCD_DrawLine(u16 x1, u16 y1, u16 x2, u16 y2);//画线
11.   void LCD_DrawLine_Color(u16 x1, u16 y1, u16 x2, u16 y2,u16 color);//指定颜色
      画线
12.   void LCD_DrowSign(uint16_t x, uint16_t y, uint16_t color);//画十字标记
13.   void LCD_DrawRectangle(u16 x1, u16 y1, u16 x2, u16 y2);//画矩形
14.   void LCD_Draw_Circle(u16 x0,u16 y0,u8 r);//画圆
15.   void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode);//显示一个字符
16.   void LCD_ShowNum(u16 x,u16 y,u32 num,u8 len,u8 size);//显示一个数字
17.   void LCD_ShowxNum(u16 x,u16 y,u32 num,u8 len,u8 size,u8 mode);//显示数字
18.   void LCD_ShowString(u16 x,u16 y,u16 width,u16 height,u8 size,u8 *p);//显示字
      符串
19.   void LCD_ShowFontHZ(u16 x, u16 y, u8 *cn);//显示汉
      字
20.   void LCD_ShowPicture(u16 x, u16 y, u16 wide, u16 high,u8 *pic);//显示图片

```

我们提供的这么多函数，其实很多函数都是调用画点函数 LCD_DrawPoint 完成的，这些函数的使用方法也很简单，我们就以显示字符函数 LCD_ShowChar 为例进行介绍（其他函数大家可参考对应的源代码）：

```

1.    //在指定位置显示一个字符
2.    //x,y:起始坐标
3.    //num:要显示的字符:" --->"~"
4.    //size:字体大小 12/16/24
5.    //mode:叠加方式(1)还是非叠加方式(0)
6.    void LCD_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode)
7.    {
8.        u8 temp,t1,t;
9.        u16 y0=y;
10.       u8 csize=(size/8+((size%8)?1:0))*(size/2);    //得到字体一个字符对应点阵
      集所占的字节数
11.       num=num-' ';//得到偏移后的值（ASCII 字库是从空格开始取模，所以-' '就是对应字符
      的字库）
12.       for(t=0;t<csize;t++)
13.       {
14.           if(size==12)temp=ascii_1206[num][t];      //调用 1206 字体
15.           else if(size==16)temp=ascii_1608[num][t]; //调用 1608 字体
16.           else if(size==24)temp=ascii_2412[num][t]; //调用 2412 字体
17.           else return;                               //没有的字库
18.           for(t1=0;t1<8;t1++)
19.           {
20.               if(temp&0x80)LCD_DrawPoint_Color(x,y,FRONT_COLOR);
21.               else if(mode==0)LCD_DrawPoint_Color(x,y,BACK_COLOR);
22.               temp<<=1;

```

```

23.         y++;
24.         if(y>=tftlcd_data.height)return;    //超区域了
25.         if((y-y0)==size)
26.         {
27.             y=y0;
28.             x++;
29.             if(x>=tftlcd_data.width)return;  //超区域了
30.             break;
31.         }
32.     }
33. }
34. }

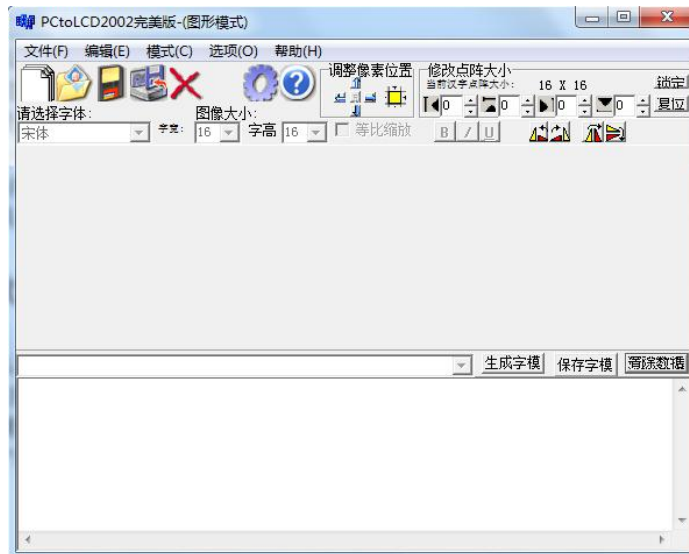
```

函数入口参数 x 和 y 用来设置显示的起始位置，num 是要显示的字符，size 用来选择显示字体大小，本实验支持 12/16/24 号的 ASCII 字符显示。mode 用来设置是否支持叠加显示，为 0 表示不使用叠加，为 1 表示使用叠加显示。叠加方式显示多用于在显示的图片上显示字符，非叠加方式一般用于普通的显示。函数内我们用到了三个字符集点阵数据数组 asc2_1206、asc2_1608、asc2_2412。

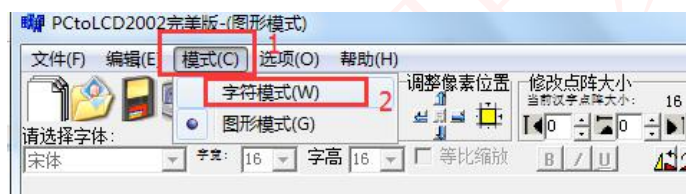
2.4.5 字符汉字图片提取软件使用

下面我们重点介绍下如何让字符显示在 TFTLCD 模块上。要显示字符，我们先要有字符的点阵数据，ASCII 常用的字符集总共有 95 个，从空格符开始，分为：! " # \$ % & ' () * + , - 0 1 2 3 4 5 6 7 8 9 : ; < = > ? @ A B C D E F G H I J K L M N O P Q R S T U V W X Y Z [\] ^ _ ` a b c d e f g h i j k l m n o p q r s t u v w x y z { | } ~ .

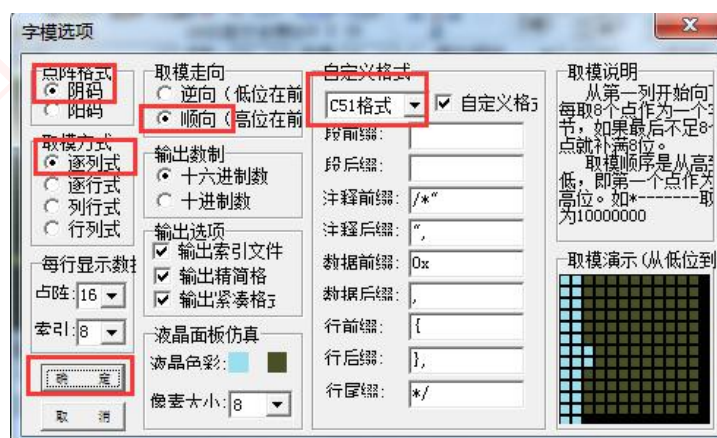
我们先要得到这个字符集的点阵数据，这里我们介绍一个款很好的字符提取软件：PCtoLCD2002 完美版，在我们光盘的“\5--开发工具\4-常用辅助开发软件\PCtoLCD2002 完美版”位置。该软件可以提供各种字符，包括汉字（字体和大小都可以自己设置）阵提取，且取模方式可以设置好几种，常用的取模方式，该软件都支持。该软件还支持图形模式，也就是用户可以自己定义图片的大小，然后画图，根据所画的图形再生成点阵数据，这功能在制作图标或图片的时候很有用。双击这个软件，弹出界面如下：



这里我们介绍下如何取汉字和字符字模数据，点击“模式”，选择“字符模式”，操作如下：



然后点击“选项”，会弹出一个字模选项对话框，将其设置为“阴码+逐列式+顺向+C51 格式”，操作如下：



上图设置的取模方式，在右上角的取模说明里面有，即：从第一列开始向下

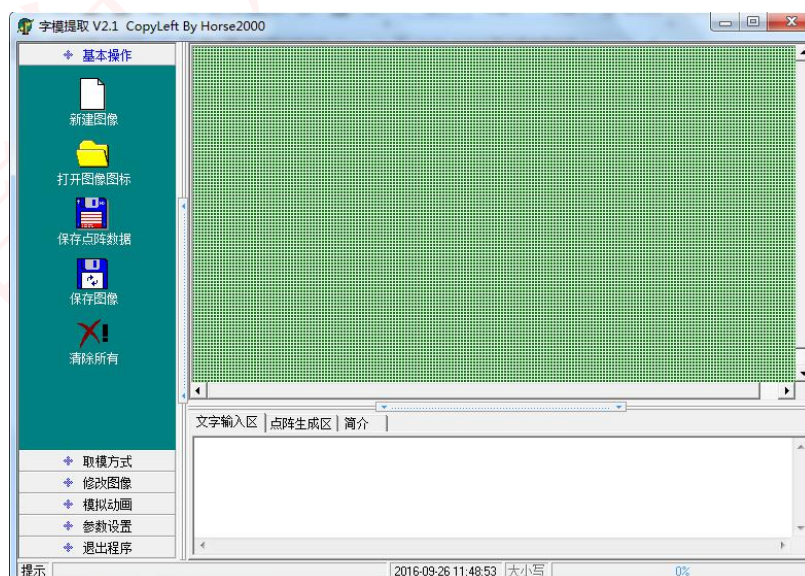
每取 8 个点作为一个字节，如果最后不足 8 个点就补满 8 位。取模顺序是从高到低，即第一个点作为最高位。如*-----取为 10000000。其实就是按如下图 的这种方式：



从上到下，从左到右，高位在前。我们按这样的取模方式，然后把 ASCII 字符集按 12*6 大小、16*8 、24*12 和 32*16 大小取模出来（对应汉字大小为 12*12、16*16 、24*24 和 32*32，字符只有汉字的一半大！），保存在 font.h 里面，每个 12*6 的字符占用 12 个字节，每个 16*8 的字符占用 16 个字节，每个 24*12 的字符占用 36 个字节。具体见 font.h 部分代码。

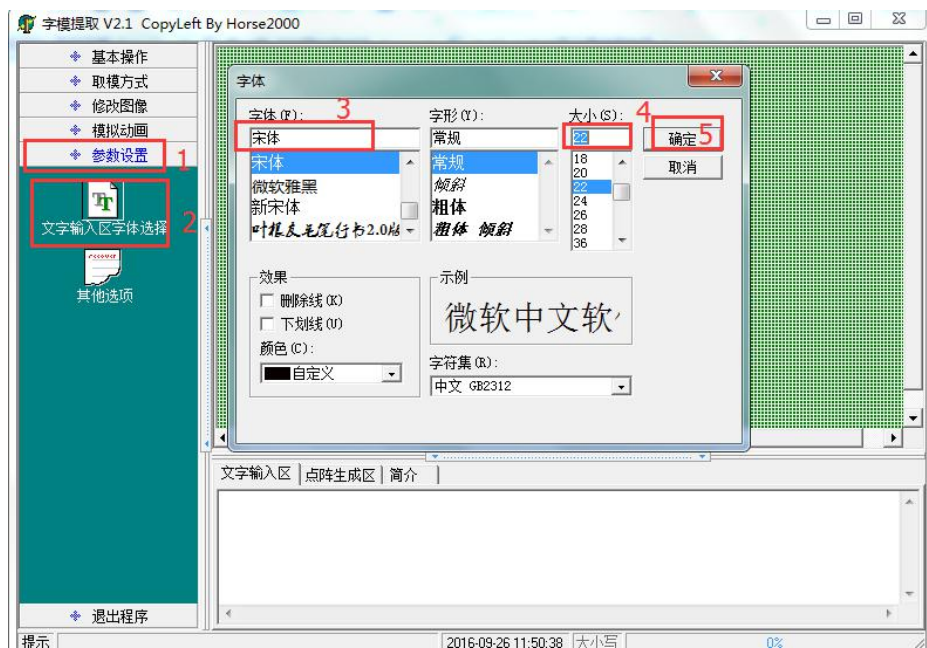
(1) 汉字显示

本实验我们还给大家提供了一个汉字显示函数，这里再给大家介绍另一款取模软件，放在光盘“\5--开发工具\4-常用辅助开发软件\文字取模软件”目录内，此软件用于汉字取模极其方便，下面打开这个软件，界面如下：

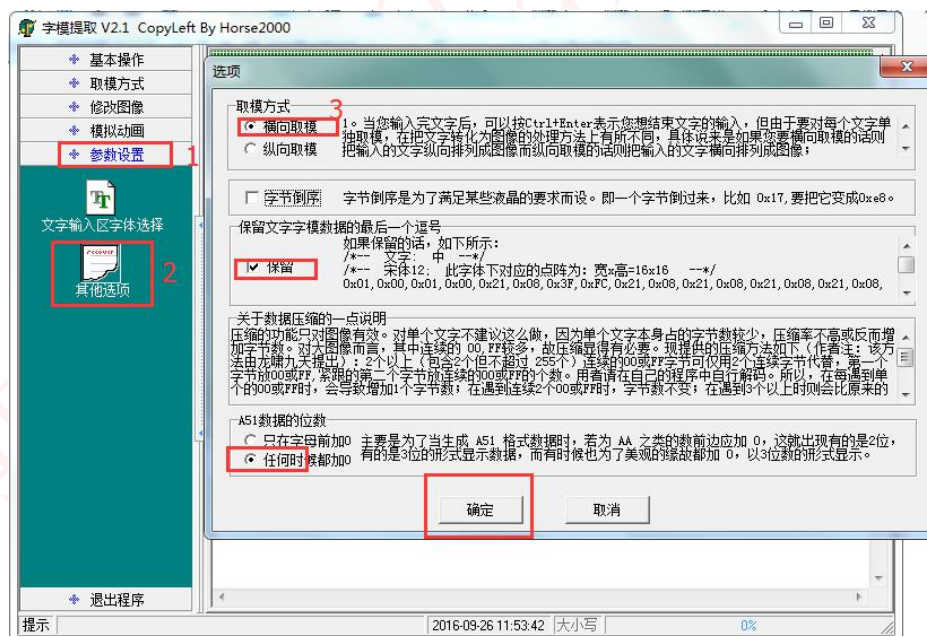


首先我们选择“参数设置”设定好字体类型及大小，本实验所取汉字大小为

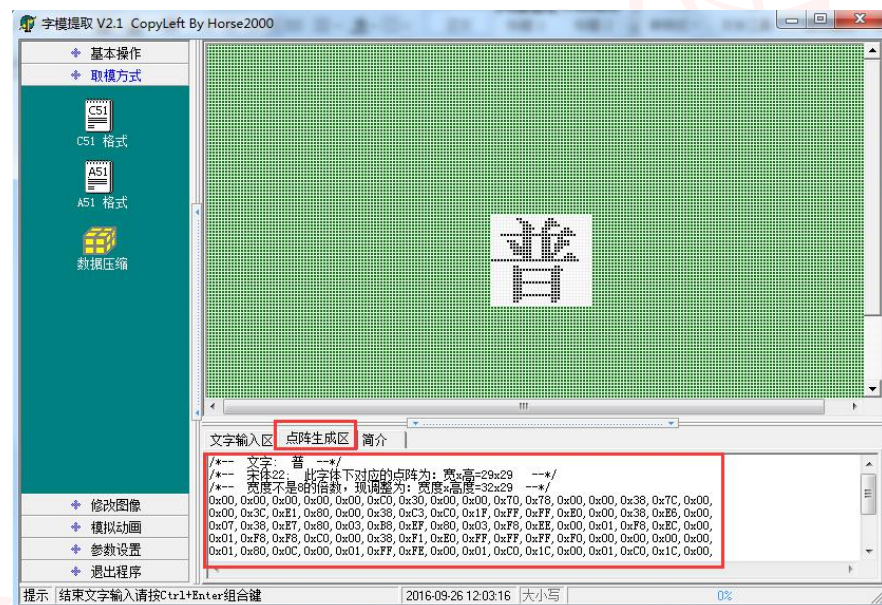
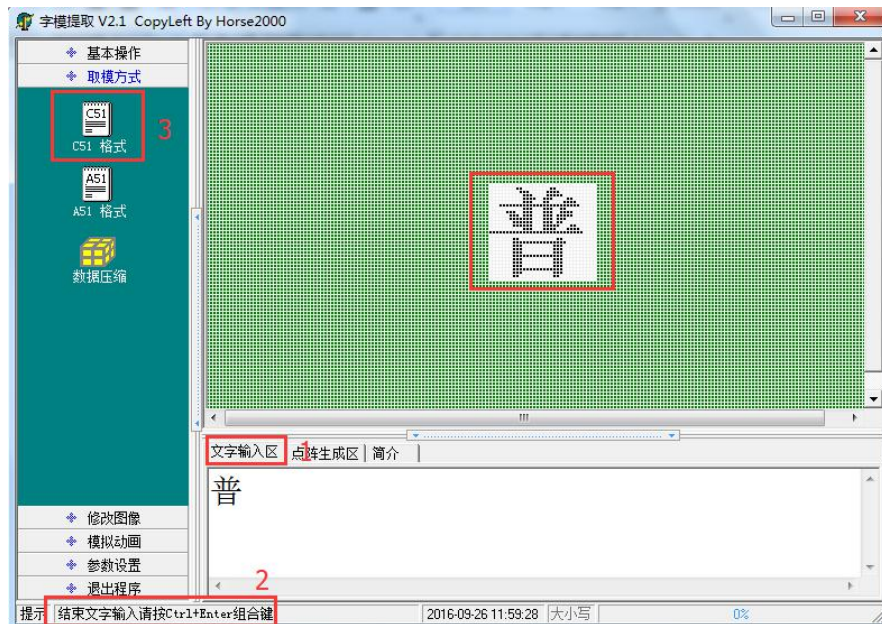
宋体 22 号，具体操作步骤如下：



设置好字体大小后，我们再来设置下取模的方式，点击“其他选项”，选择“横向取模”，操作步骤如下：



设置好取模方式后，接下来就是取汉字字模数据了，我们直接在“文字输入区”输入所要显示的汉字，比如输入“普”，然后按下“Ctrl+Enter 组合键”就会在窗口显示输入的汉字，点击“取模方式”选择“C51 格式”，就会在“点阵生成区”显示汉字“普”的字模数据，操作步骤如下：



这样我们直接将其保存在 font.h 的 CnChar32x29 数组内，我们在定义显示汉字时还加了汉字内锁码，所以在保存这些汉字字模数据时，前面加上我们所输入的汉字“普”，如下：

```

330 "普", 0x00, 0x00, 0x00, 0x00, 0x00, 0xC0, 0x30, 0x00, 0x00, 0x70, 0x78, 0x00, 0x00, 0x38, 0x7C, 0x00,
331 0x00, 0x3C, 0xE1, 0x80, 0x00, 0x38, 0xC3, 0xC0, 0x1F, 0xFF, 0xFF, 0xE0, 0x00, 0x38, 0xE6, 0x00,
332 0x07, 0x38, 0xE7, 0x80, 0x03, 0xB8, 0xEF, 0x80, 0x03, 0xF8, 0xEE, 0x00, 0x01, 0xF8, 0xEC, 0x00,
333 0x01, 0xF8, 0xF8, 0xC0, 0x00, 0x38, 0xF1, 0xE0, 0xFF, 0xFF, 0xFF, 0xF0, 0x00, 0x00, 0x00, 0x00,
334 0x01, 0x80, 0x0C, 0x00, 0x01, 0xFF, 0xFE, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xC0, 0x1C, 0x00,
335 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xFF, 0xFC, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xC0, 0x1C, 0x00,
336 0x01, 0xC0, 0x1C, 0x00, 0x01, 0xFF, 0xFC, 0x00, 0x01, 0xC0, 0x1C, 0x00, 0x01, 0x80, 0x18, 0x00,
337 0x00, 0x00, 0x00, 0x00,

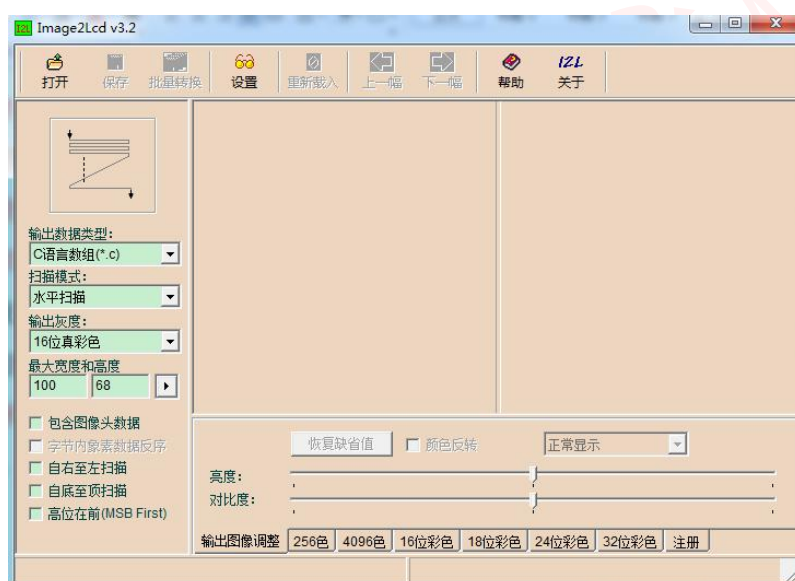
```

如果要显示其他的汉字，取模方法和上述一样，这些显示函数的具体实现过程，大家可以参考工程源代码，这里就不多介绍。

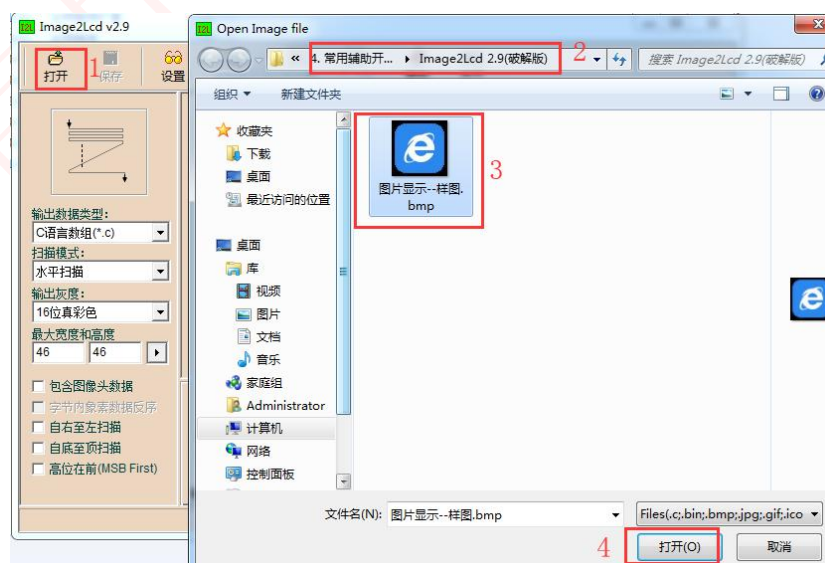
注意：有的朋友在使用取模软件取汉字时会出现这样一种情况，明明是按照教程来设置取模软件参数，但是最后取的汉字数据不是 16*16 大小的，这种情况是因为你电脑系统内缺少某种字库，你可以换台电脑测试，如果可以就把系统内字库拷贝到你电脑中去，怎么查找系统字库，这个就需要自己百度了。

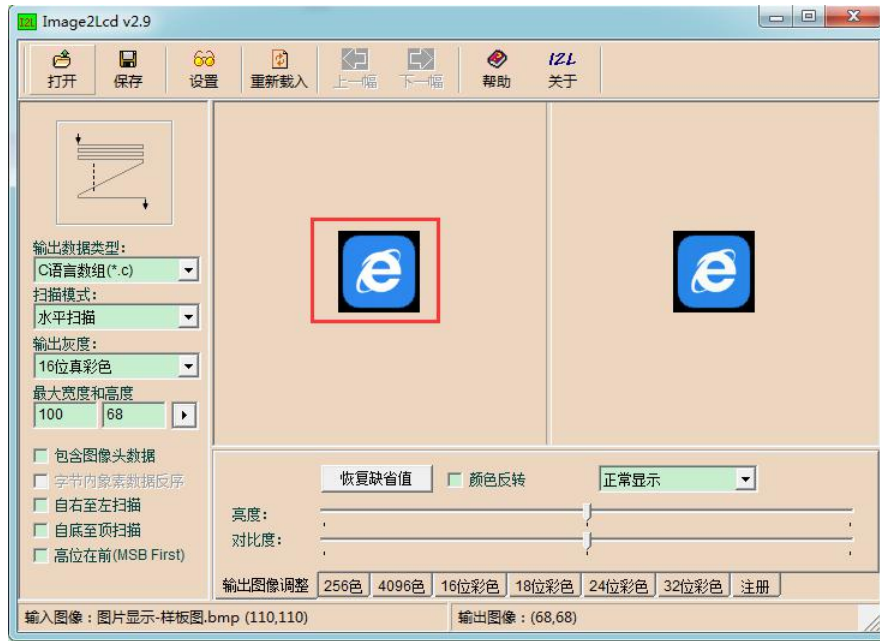
(2) 图片显示

在我们的 tftlcd.c 文件内还提供了图片显示函数，那么怎么来显示图片呢？这里再给大家介绍另一款取模软件，放在光盘“\5--开发工具\4-常用辅助开发软件\Image2Lcd 2.9(破解版)”目录内，此软件用于图片取模极其方便，打开后界面如下：

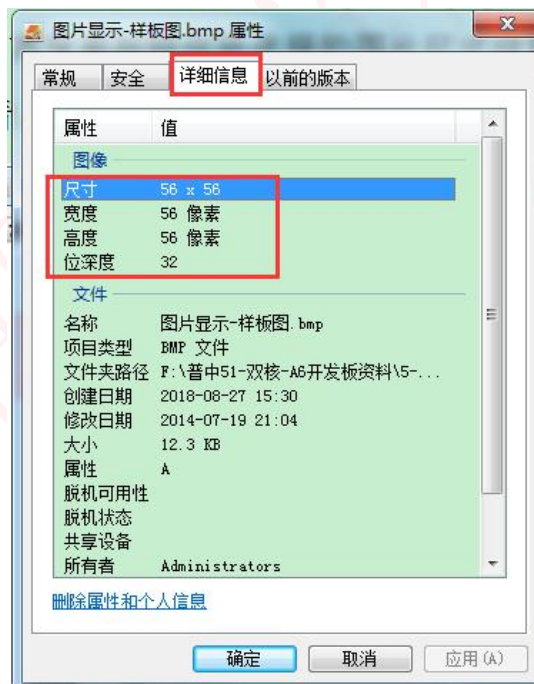


1，点击“打开”菜单按钮，选择你想要显示的图片，注意图片格式要是. BMP，如下所示：

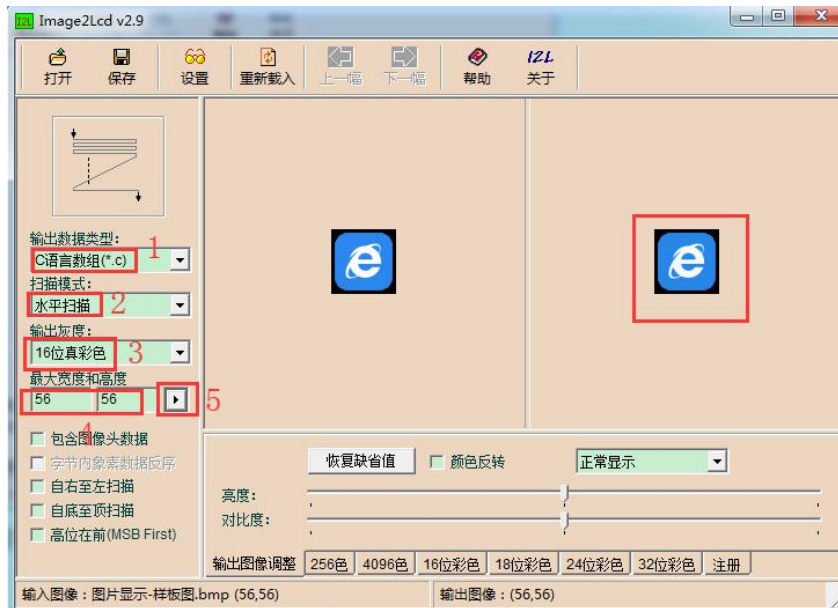




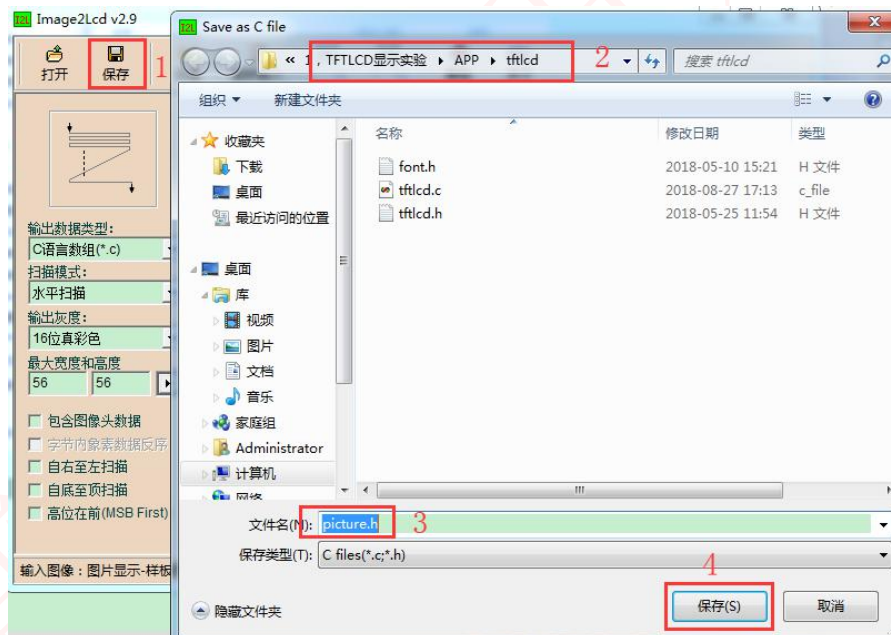
2, 设置相关参数, 包括输出数据类型、扫描模式、输出灰度、最大宽度和高度等, **注意: 最大宽度和高度要按照你所选择的图片尺寸设置。** 图片的实际尺寸大小可点击图片属性查看, 如下:



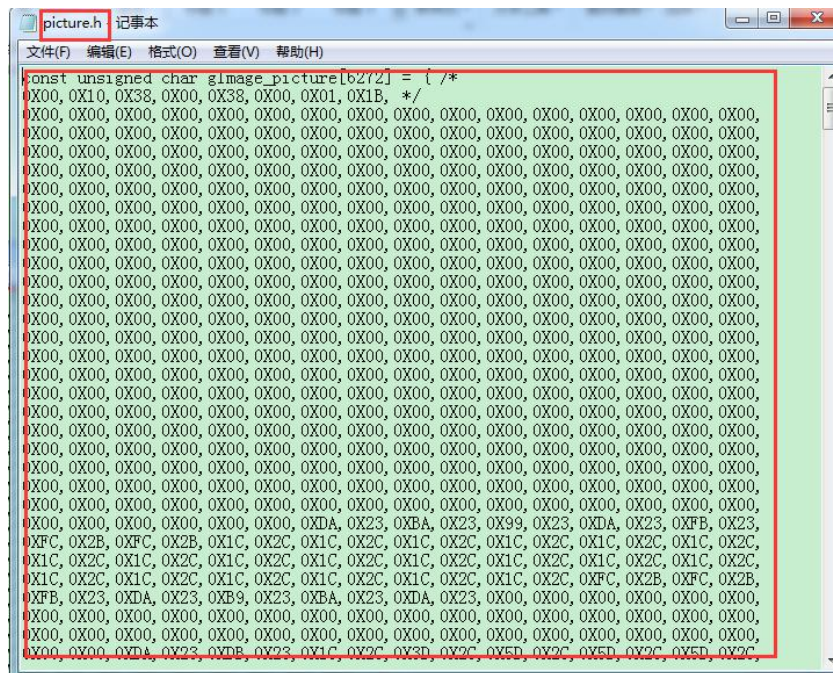
具体设置如下: 其他未标注的保持默认即可。



4, 点击“保存”菜单按钮, 将输出的图片数据保存为 xxx.h 格式, 存储到 tftlcd 文件夹内, 这里我们就命名为 picture.h, 如下:



保存好后就会弹出图片取模后的数据, 如下:



上述 gImage_picture 数组即为该图片的数据。

然后在 main.c 文件内将 picture.h 头文件包含进来并且主函数内调用显示图片函数即可，如下：

```
LCD_ShowPicture(10, 60, 56, 56, gImage_picture);
```

函数中前面两个参数 10, 60 为图片显示的起始位置，后面两个参数 56, 56 分别是图片的宽度和高度，这两个值一定要保证和前面取模时图片大小一致，否则显示出错。最后一个参数 gImage_picture 即为存储图片数据的数组。

2.4.6 主函数

编写好 TFTLCD 初始化和显示函数后，接下来就可以编写主函数了，代码如下：

```
1. #include "system.h"
2. #include "SysTick.h"
3. #include "led.h"
4. #include "usart.h"
5. #include "tftlcd.h"
6. #include "picture.h"
7.
8.
9. /*****
   ****
```

```

10. * 函数名      : main
11. * 函数功能    : 主函数
12. * 输 入      : 无
13. * 输 出      : 无
14. ****/
15. int main()
16. {
17.     u8 i=0;
18.     u16 color=0;
19.
20.     SysTick_Init(72);
21.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
22.     LED_Init();
23.     USART1_Init(115200);
24.     TFTLCD_Init(); //LCD 初始化
25.
26.     FRONT_COLOR=BLACK;
27.     LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,12,"Hello Worl
d!");
28.     LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"Hello Worl
d!");
29.     LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,24,"Hello Worl
d!");
30.     LCD_ShowFontHZ(10, 80,"普中科技");
31.     LCD_ShowString(10,120,tftlcd_data.width,tftlcd_data.height,24,"www.prech
in.cn");
32.
33.     LCD_Fill(10,150,60,180,GRAY);
34.     color=LCD_ReadPoint(20,160);
35.     LCD_Fill(100,150,150,180,color);
36.     printf("color=%x\r\n",color);
37.
38.     LCD_ShowPicture(20,220,200,112,(u8 *)gImage_picture);
39.
40.     while(1)
41.     {
42.         i++;
43.         if(i%20==0)
44.         {
45.             LED1=!LED1;
46.         }
47.
48.         delay_ms(10);

```

```
49.     }  
50. }
```

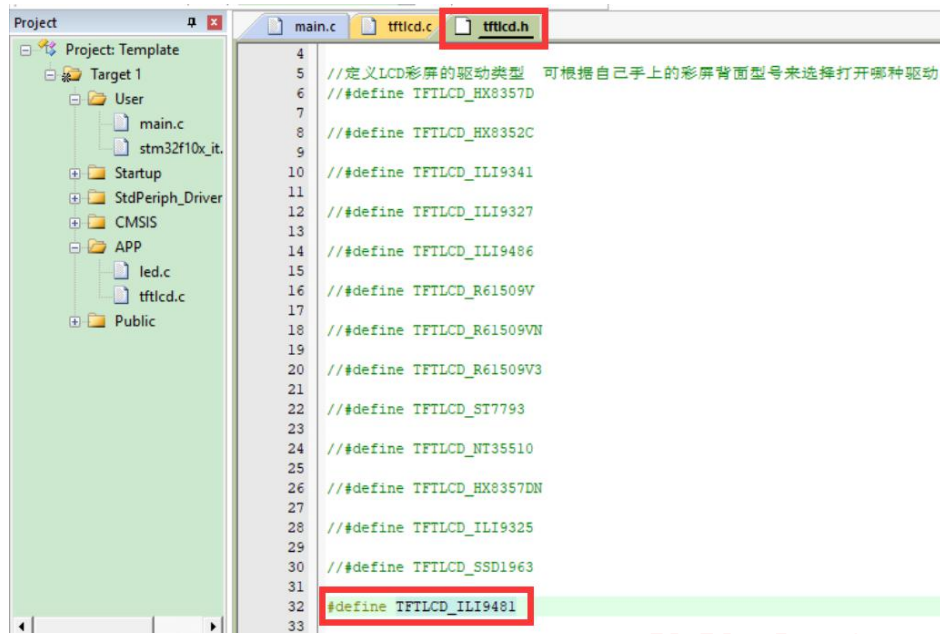
主函数实现的功能很简单，首先调用之前编写好的硬件初始化函数，包括 SysTick 系统时钟，LED 初始化等。然后调用我们前面编写的 TFTLCD_Init 函数，用来初始化 TFTLCD，默认设置为竖屏显示，背景颜色为白色。设置显示颜色为黑色后，调用字符串及汉字显示函数进行显示，最后进入 while 循环，控制 D1 指示灯会间隔 200ms 闪烁，提示系统正常运行。

2.5 实验现象

将工程程序编译后下载到开发板内，可以看到 D1 指示灯不断闪烁，表示程序正常运行。将 TFTLCD 模块插上开发板上的彩屏接口，按下复位键后，TFTLCD 即可显示字符及汉字信息。如下图所示：



实验说明：下载有关 TFTLCD 显示实验时，请确保您所使用的 TFTLCD 彩屏驱动型号与程序内选择的一致，如何查看呢？首先看下您 TFTLCD 彩屏板左上角对应的驱动型号，比如 ILI9481。然后打开实验程序，查看 tftlcd.h 头文件开始处对应的 TFTLCD 驱动型号选择的是哪个？如下所示：



课后作业

- (1) 在本实验基础上实现其他大小的 ASCII 或汉字显示。（温馨提示：按照前面所介绍的取模软件使用方法取模，然后在驱动程序内修改即可）
- (2) 可以将前面章节的实验，采用 TFTLCD 彩屏显示，就不需要 printf 打印输出了。

第 3 章 触摸屏实验

在前面章节我们介绍了使用 TFTLCD 模块显示字符和汉字,利用 TFTLCD 模块,STM32 系统就有了高级信息输出的功能,如果我们还希望有一个友好的用户输入的设备,触摸屏就是非常好的选择,现如今大多电子产品是将触摸屏配合液晶显示器组成人机交互系统,比如手机、平板等。因此我们很有必要学习下触摸屏的控制。这一章我们来学习下如何使用 STM32F1 来驱动触摸屏。我们 STM32F1 开发板本身并没有触摸屏控制器,但是它支持触摸屏,可以通过外接带触摸屏的 LCD 模块(比如我们的 TFTLCD 模块)来实现触摸屏控制。本章要实现的功能是:通过 TFTLCD 模块上的触摸板(包括电阻触摸和电容触摸)实现触摸功能,最终实现一个画板的功能。学习本章可以参考“\6--芯片资料\彩屏数据手册”相关芯片数据手册(电阻触摸屏和电容触摸屏)。本章分为如下几部分内容:

- 3.1 触摸屏介绍
- 3.2 硬件设计
- 3.3 软件设计
- 3.4 实验现象

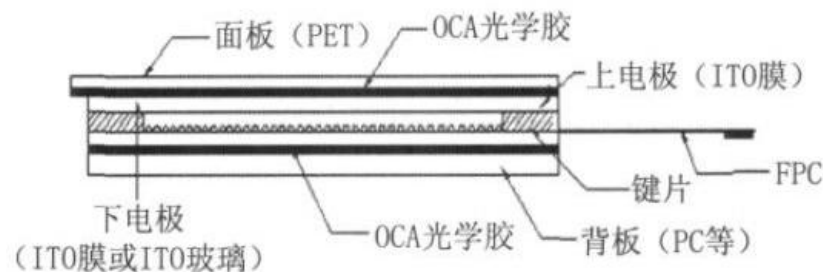
3.1 触摸屏介绍

触摸屏又称触控面板，它是一种把触摸位置转化成坐标数据的输入设备，根据触摸屏的检测原理，主要分为电阻式触摸屏和电容式触摸屏。下面我们就分别来介绍下这两种触摸屏。

3.1.1 电阻式触摸屏介绍

电阻式触摸屏是一种传感器，它将矩形区域中触摸点(X, Y)的物理位置转换为代表 X 坐标和 Y 坐标的电压。很多 LCD 模块都采用了电阻式触摸屏，比如我们 2.0/2.2/2.4/2.6/2.8/3.2/3.5/4.3/7.0 寸的 TFTLCD 模块都是采用电阻式触摸屏。使用时需要用一定的压力才会能检测到电压，即触摸。

电阻式触摸屏基本上是薄膜加上玻璃的结构，薄膜和玻璃相邻的一面上均涂有 ITO(纳米铟锡金属氧化物)涂层，ITO 具有很好的导电性和透明性。当触摸操作时，薄膜下层的 ITO 会接触到玻璃上层的 ITO，经由感应器传出相应的电信号，经过转换电路送到处理器，通过运算转化为屏幕上的 X、Y 值，而完成点选的动作，并呈现在屏幕上。电阻式触摸屏结构如下图所示：



电阻触摸屏的工作原理主要是通过压力感应原理来实现对屏幕内容的操作和控制的，这种触摸屏屏体部分是一块与显示器表面非常配合的多层复合薄膜，其中第一层为玻璃或有机玻璃底层，第二层为隔层，第三层为多元树脂表层，表面还涂有一层透明的导电层，上面再盖有一层外表面经硬化处理、光滑防刮的塑料层。在多元脂表层表面的传导层及玻璃层感应器是被许多微小的隔层所分隔电流通过表层，轻触表层压下时，接触到底层，控制器同时从四个角读出相称的电流及计算手指位置的距离。这种触摸屏利用两层高透明的导电层组成触摸屏，两层之间距离仅为 2.5 微米。当手指触摸屏幕时，平常相互绝缘的两层导电层就在

触摸点位置有了一个接触，因其中一面导电层接通 Y 轴方向的电源均匀电压场，使得侦测层的电压由零变为非零，控制器侦测到这个接通后，进行 A/D 转换，并将得到的电压值与参考电压相比，即可得触摸点的 Y 轴坐标，同理得出 X 轴的坐标，这就是所有电阻技术触摸屏共同的最基本原理。

电阻触摸屏的优点：精度高、价格便宜、抗干扰能力强、稳定性好。

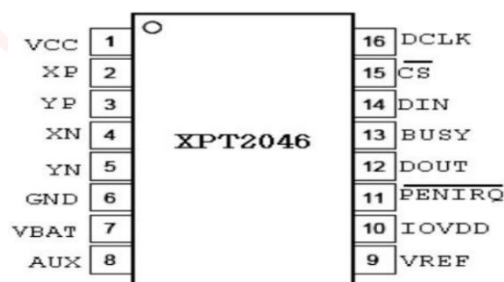
电阻触摸屏的缺点：容易被划伤、透光性不太好、不支持多点触摸。

从上面的简介，我们知道触摸屏都需要一个 AD 转换器，也就是要将电压变化读取出来，供主机求出触摸的位置。我们的 TFTLCD 模块使用的是四线电阻式触摸屏，这种触摸屏的控制芯片有很多，包括：ADS7843、ADS7846、TSC2046、XPT2046 和 AK4182 等。这几款芯片的驱动基本上是一样的，也就是说你只要写出了 XPT2046 的驱动，这个驱动对其他几个芯片也是有效的。而且封装也有一样的，而且管脚也完全兼容。所以在替换起来非常方便。

我们彩屏上面使用的触摸屏控制芯片是 XPT2046。XPT2046 的特点主要有：

- 1) 一款 4 导线制触摸屏控制器，采用 SPI 模式进行通信。
- 2) 内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。
- 3) 支持从 1.5V 到 5.25V 的低电压 I/O 接口。
- 4) 只需执行两次 A/D 转换即可查出被按的屏幕位置。
- 5) 可以测量加在触摸屏上的压力
- 6) 芯片内部自带温度检测、电池电压（0-6V）监测等等

XPT2046 SOP 封装有 16 个引脚，如下图所示：



其引脚说明如下图所示：

QFN引脚号	TSSOP引脚号	VFBGA引脚号	名称	说明
1	13	A5	BUSY	忙时信号线。当CS为高电平时为高阻状态
2	14	A4	DIN	串行数据输入端。当CS为低电平时，数据在DCLK上升沿锁存进来
3	15	A3	CS	片选信号。控制转换时序和使能串行输入输出寄存器，高电平时ADC掉电
4	16	A2	DCLK	外部时钟信号输入
5	1	B1和C1	VCC	电源输入端
6	2	D1	XP	XP位置输入端
7	3	E1	YP	YP位置输入端
8	4	G2	XN	XN位置输入端
9	5	G3	YN	YN位置输入端
10	6	G4和G5	GND	接地
11	7	G6	VBAT	电池监视输入端
12	8	E7	AUX	ADC辅助输入通道
13	9	D7	VREF	参考电压输入/输出
14	10	C7	IOVDD	数字电源输入端
15	11	B7	PENIRQ	笔接触中断引脚
16	12	A6	DOUT	串行数据输出端。数据在DCLK的下降沿移出，当CS高电平时为高阻状态

XPT2046 芯片详细的介绍可以参考“\6--芯片资料\开发板芯片数据手册”
《XPT2046》。

3.1.2 电容式触摸屏介绍

现在几乎所有智能手机，包括平板电脑都是采用电容屏作为触摸屏，电容屏是利用人体感应进行触点检测控制，不需要直接接触或只需要轻微接触，通过检测感应电流来定位触摸坐标。

我们有的 3.5/4.3/4.5/7 寸 TFTLCD 模块上使用的触摸屏是电容式触摸屏，下面简单介绍下电容式触摸屏的原理。

电容式触摸屏主要分为两种：

(1) 表面电容式电容触摸屏。

表面电容式触摸屏技术是利用 ITO(铟锡氧化物，是一种透明的导电材料)导电膜，通过电场感应方式感测屏幕表面的触摸行为进行。但是表面电容式触摸屏有一些局限性，它只能识别一个手指或者一次触摸。

(2) 投射式电容触摸屏。

投射式电容触摸屏却具有多指触控的功能。这两种电容式触摸屏都具有透光率高、反应速度快、寿命长等优点，缺点是：随着温度、湿度的变化，电容值会发生变化，导致工作稳定性差，时常会有漂移现象，需要经常校对屏幕，且不可佩戴普通手套进行触摸定位。

投射电容式触摸屏是传感器利用触摸屏电极发射出静电场线。一般用于投射电容传感技术的电容类型有两种：自我电容和交互电容。

自我电容又称绝对电容，是最广为采用的一种方法，自我电容通常是指扫描电极与地构成的电容。在玻璃表面有用 ITO 制成的横向与纵向的扫描电极，这些电极和地之间就构成一个电容的两极。当用手或触摸笔触摸的时候就会并联一个电容到电路中去，从而使在该条扫描线上的总体的电容量有所改变。在扫描的时候，控制 IC 依次扫描纵向和横向电极，并根据扫描前后的电容变化来确定触摸点坐标位置。笔记本电脑触摸输入板就是采用的这种方式，笔记本电脑的输入板采用 X*Y 的传感电极阵列形成一个传感格子，当手指靠近触摸输入板时，在手指和传感电极之间产生一个小量电荷。采用特定的运算法则处理来自行、列传感器的信号来确定手指的位置。

交互电容又叫做跨越电容，它是在玻璃表面的横向和纵向的 ITO 电极的交叉处形成电容。交互电容的扫描方式就是扫描每个交叉处的电容变化，来判定触摸点的位置。当触摸的时候就会影响到相邻电极的耦合，从而改变交叉处的电容量，交互电容的扫描方法可以侦测到每个交叉点的电容值和触摸后电容变化，因而它需要的扫描时间与自我电容的扫描方式相比要长一些，需要扫描检测 X*Y 根电极。目前智能手机/平板电脑等的触摸屏，都是采用交互电容技术。

我们使用的电容屏也是采用投射式电容屏（交互电容类型），所以后面仅以投射式电容屏作为介绍。

投射式电容触摸屏（交互电容类型）内部由驱动电极与接收电极组成，驱动电极发出低电压高频信号投射到接收电极形成稳定的电流，当人体接触到电容屏时，由于人体接地，手指与电容屏就形成一个等效电容，而高频信号可以通过这一等效电容流入地线，这样，接收端所接收的电荷量减小，而当手指越靠近发射端时，电荷减小越明显，最后根据接收端所接收的电流强度来确定所触碰的点。

以上就是电容触摸屏的基本原理，更多详细的资料大家可以百度了解下。

电容触摸屏通常也需要一个驱动 IC 来检测电容触摸，且一般是通过 I2C 接口输出触摸数据的。我们 4.3 寸电容触摸屏使用的驱动 IC 是 FT5336，不清楚自己彩屏驱动芯片及驱动触摸 IC 型号的可以看下彩屏背面的型号。FT5336 支持最多 5 点触摸，这里我们以 FT5336 为例进行介绍，其他的驱动 IC 参考着学习。

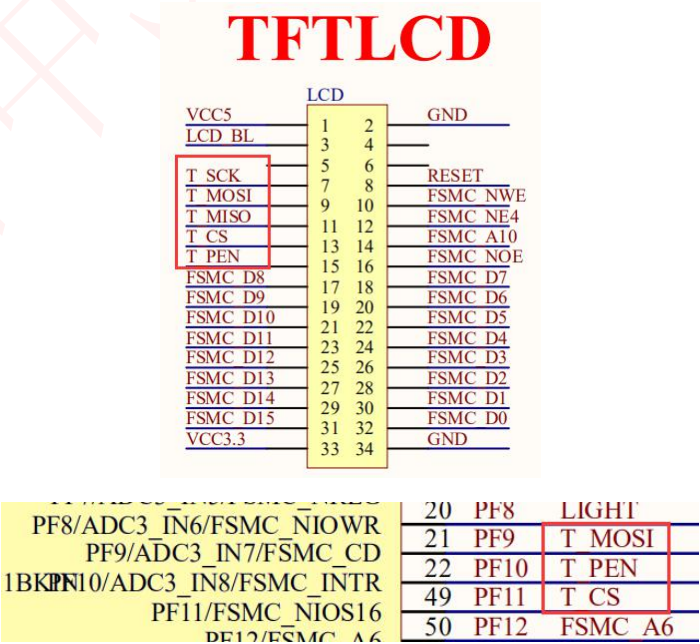
FT5336 是台湾敦泰電子股份有限公司生产的一颗电容触摸屏驱动 IC，最多支持 13*24 (314) 个通道。支持 SPI/IIC 接口，我们使用的就是 IIC 接口。IIC 接口模式下，该驱动 IC 与 STM32F1 的连接仅需要 4 根线：SDA、SCL、RST 和 INT，SDA 和 SCL 是 IIC 通信用的，RST 是复位脚（低电平有效），INT 是中断输出信号，关于 IIC 我们就不详细介绍了，在前面 I2C 实验时已经讲过。关于 FT5336 或者其他电容触摸驱动 IC 详细内容，大家可以在光盘“\6--芯片资料\彩屏数据手册”内查看。这里不多说，我们主要还是看代码，从代码中体会它们的使用。

3.2 硬件设计

本实验使用到硬件资源如下：

- (1) DS0 指示灯
- (2) KEY_UP 按键
- (3) TFTLCD 模块（带电阻或电容式触摸屏）
- (4) AT24C02

DS0 指示灯、KEY_UP 按键、TFTLCD 模块（彩屏显示部分）、AT24C02 电路在前面章节都介绍过，这里就不多说，下面我们看下 TFTLCD 模块上的触摸屏（电阻屏和电容屏）与 STM32F1 的连接图，如下图所示：



T_MISO	T_SCK	PB1	47	PB0/ADC12_IN8/TIM5_CH3/TIM8_CH2N
BOOT1	BOOT1	PB2	48	PB1/ADC12_IN9/TIM3_CH4/TIM8_CH3N
PF9	PF9	PB2	122	PB2/BOOT1

从电路图中可以看到，T_MOSI、T_MISO、T_SCK、T_CS 和 T_PEN 分别连接在 STM32F1 的 PF9、PB2、PB1、PF11 和 PF10 上。而 FSMC 相关引脚之前我们介绍 TFTLCD 实验的时候就给大家介绍。如果是电阻触摸屏的话，它是直接连接在 XPT2046 触摸芯片 SPI 接口及笔中断引脚上的。如果是电容触摸屏的话，只需要 4 根线即可，分别是 T_PEN(INT)、T_CS(RST)、T_CLK(SCL) 和 T_MOSI(SDA)。其中：INT、RST、SCL 和 SDA 分别是 FT5336 的中断输出信号、复位信号，IIC 的 SCL 和 SDA 信号。这里我们用查询的方式读取 FT5336 的数据，没有用到中断信号（INT），所以同 STM32F1 的连接，只需要 3 根线即可，不过我们还是预留 INT 那根，方便扩展的其他驱动 IC 做 IIC 地址设定，所以保持 4 根线连接。

DS0 指示灯用来提示系统运行状态，KEY_UP 按键用来强制校准电阻触摸屏（电容触摸屏无需校准），如果出现触摸不准，可以通过此按键强制校准。AT24C02 用来存储电阻触摸屏校准数据，TFTLCD 模块（带电阻或电容触摸屏）用来显示触摸。

3.3 软件设计

本章所要实现的功能是：通过 TFTLCD 模块上的触摸板（包括电阻触摸和电容触摸）实现触摸功能，最终实现一个画板的功能。对于电阻触摸屏，当出现触摸不准，可使用 KEY_UP 键校准，校准参数存储在 24C02 内。程序框架如下：

- （1）初始化触摸屏
- （2）编写触摸（电阻/电容）扫描函数
- （3）编写电阻触摸校准函数
- （4）编写主函数

电阻触摸屏采用的是 SPI 通信（使用 IO 口模拟 SPI 时序），电容触摸屏采用的是 IIC 通信，这些通信方式在前面都介绍。下面我们打开“\4--实验程序\1--基础实验\35-触摸屏实验”工程，在 APP 工程组中可以看到添加了 touch.c、ctiic.c、gt5663.c 文件（里面包含了电阻屏/电容屏驱动程序），同时还要包含对应的头文件路径。

这里我们分析几个重要函数，其他部分程序大家可以打开工程查看。

3.3.1 触摸屏初始化函数

要使用触摸屏，我们必须先对它所使用的 IO 进行配置。初始化代码如下：

```
1. //触摸屏初始化
2. //返回值:0,进行过校准
3. //      1,未进行校准
4. u8 TP_Init(void)
5. {
6.     #if defined(TFTLCD_HX8357D)||defined(TFTLCD_ILI9341)||defined(TFTLCD_HX8352C)|| \
7.         defined(TFTLCD_R61509V)||defined(TFTLCD_R61509VN)||defined(TFTLCD_R61509V3)|| \
8.         defined(TFTLCD_ILI9486)||defined(TFTLCD_ST7793)||defined(TFTLCD_ILI9325)|| \
9.         defined(TFTLCD_ILI9327)||defined(TFTLCD_ILI9481)||defined(TFTLCD_SSD1963)
10.
11.     GPIO_InitTypeDef GPIO_InitStructure;
12.
13.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOF, ENABLE); //使能 PB,PF
        端口时钟
14.
15.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1; // PB1 端口配置
16.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
17.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
18.     GPIO_Init(GPIOB, &GPIO_InitStructure); //B1 推挽输出
19.     GPIO_SetBits(GPIOB,GPIO_Pin_1); //上拉
20.
21.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2; // PB2 端口配置
22.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; //上拉输入
23.     GPIO_Init(GPIOB, &GPIO_InitStructure); //B2 上拉输入
24.     GPIO_SetBits(GPIOB,GPIO_Pin_2); //上拉
25.
26.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11|GPIO_Pin_9; // F9, PF11 端口配置
27.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
28.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
29.     GPIO_Init(GPIOF, &GPIO_InitStructure); //PF9,PF11 推挽输出
30.     GPIO_SetBits(GPIOF, GPIO_Pin_11|GPIO_Pin_9); //上拉
31.
32.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10; // PF10 端口配置
33.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; //上拉输入
34.     GPIO_Init(GPIOF, &GPIO_InitStructure); //PF10 上拉输入
35.     GPIO_SetBits(GPIOF,GPIO_Pin_10); //上拉
36.
37.     TP_Read_XY(&tp_dev.x[0],&tp_dev.y[0]); //第一次读取初始化
38.     AT24CXX_Init(); //初始化 24CXX
39.     if(TP_Get_Adjdata())return 0; //已经校准
```



```

40.     else                //未校准?
41.     {
42.         LCD_Clear(WHITE); //清屏
43.         TP_Adjust();      //屏幕校准
44.         TP_Save_Adjdata();
45.     }
46.     TP_Get_Adjdata();
47. #endif
48.
49. #if defined(TFTLCD_NT35510)
50.     GT5663_Init();
51.     return 0;
52. #endif
53.     return 1;
54. }

```

在 TP_Init() 函数中，首先通过 TFTLCD 的彩屏型号宏定义标识符来选择编译电阻触摸屏还是电容触摸屏驱动程序。

使能触摸屏 IO 口时钟，其次配置对应 IO 口模式，并初始化 GPIO。因为在初始化触摸屏时，我们需要判断是否经过校准，校准的参数保存在 AT24C02 内，所以还需要初始化 24C02，并调用函数 TP_Get_Adjdata 从 24C02 的 213 地址内读取校准状态 tempfac。如果 tempfac 不等于 0X0A，执行校准函数 TP_Adjust，并保存校准数据到 24C02 中。

tp_dev 结构体用来保存校准因数和状态等，其具体成员在 touch.h 文件内定义了。

如果是电容触摸屏，其不需要校准，直接调用 GT5663_Init 函数实现初始化配置。

3.3.2 触摸屏校准函数

初始化函数比较简单，下面我们重点看下触摸屏的校准函数（触摸校准只针对电阻屏）。代码如下：

```

1.     //触摸屏校准代码
2.     //得到四个校准参数
3.     void TP_Adjust(void)
4.     {
5.         #if defined(TFTLCD_HX8357D) || defined(TFTLCD_ILI9341) || defined(TFTLCD_HX8352C) || \
6.             defined(TFTLCD_R61509V) || defined(TFTLCD_R61509VN) || defined(TFTLCD_R61509V3) || \

```

```

7.     defined(TFTLCD_ILI9486)||defined(TFTLCD_ST7793)||defined(TFTLCD_ILI9325)|| \
8.     defined(TFTLCD_ILI9327)||defined(TFTLCD_ILI9481)||defined(TFTLCD_SSD1963)
9.
10.    u16 pos_temp[4][2]; //坐标缓存值
11.    u8  cnt=0;
12.    u16 d1,d2;
13.    u32 tem1,tem2;
14.    double fac;
15.    u16 outtime=0;
16.    cnt=0;
17.    FRONT_COLOR=BLUE;
18.    BACK_COLOR =WHITE;
19.    LCD_Clear(WHITE); //清屏
20.    FRONT_COLOR=RED; //红色
21.    LCD_Clear(WHITE); //清屏
22.    FRONT_COLOR=BLACK;
23.    LCD_ShowString(40,40,160,100,16,(u8*)TP_REMIND_MSG_TBL); //显示提示信息
24.    TP_Drow_Touch_Point(20,20,RED); //画点 1
25.    tp_dev.sta=0; //消除触发信号
26.    tp_dev.xfac=0; //xfac 用来标记是否校准过,所以校准之前必须清掉! 以免错误
27.    while(1) //如果连续 10 秒钟没有按下,则自动退出
28.    {
29.        tp_dev.scan(1); //扫描物理坐标
30.        if((tp_dev.sta&0xc0)==TP_CATH_PRES) //按键按下了一次(此时按键松开了.)
31.        {
32.            outtime=0;
33.            tp_dev.sta&=~(1<<6); //标记按键已经被处理过了.
34.
35.            pos_temp[cnt][0]=tp_dev.x[0];
36.            pos_temp[cnt][1]=tp_dev.y[0];
37.            cnt++;
38.            switch(cnt)
39.            {
40.                case 1:
41.                    TP_Drow_Touch_Point(20,20,WHITE); //清除点 1
42.                    TP_Drow_Touch_Point(tftlcd_data.width-20,20,RED); //画点 2
43.                    break;
44.                case 2:
45.                    TP_Drow_Touch_Point(tftlcd_data.width-20,20,WHITE); //清除点 2
46.                    TP_Drow_Touch_Point(20,tftlcd_data.height-20,RED); //画点 3
47.                    break;
48.                case 3:
49.                    TP_Drow_Touch_Point(20,tftlcd_data.height-20,WHITE); //清除点 3

```

```

50.          TP_Drow_Touch_Point(tftlcd_data.width-20,tftlcd_data.height-20,RED);
    //画点 4
51.          break;
52.      case 4:  //全部四个点已经得到
53.          //对边相等
54.          tem1=abs(pos_temp[0][0]-pos_temp[1][0]); //x1-x2
55.          tem2=abs(pos_temp[0][1]-pos_temp[1][1]); //y1-y2
56.          tem1*=tem1;
57.          tem2*=tem2;
58.          d1=sqrt(tem1+tem2); //得到 1,2 的距离
59.
60.          tem1=abs(pos_temp[2][0]-pos_temp[3][0]); //x3-x4
61.          tem2=abs(pos_temp[2][1]-pos_temp[3][1]); //y3-y4
62.          tem1*=tem1;
63.          tem2*=tem2;
64.          d2=sqrt(tem1+tem2); //得到 3,4 的距离
65.          fac=(float)d1/d2;
66.          if(fac<0.95||fac>1.05||d1==0||d2==0) //不合格
67.          {
68.              cnt=0;
69.              TP_Drow_Touch_Point(tftlcd_data.width-20,tftlcd_data.height-20,WHIT
E); //清除点 4
70.              TP_Drow_Touch_Point(20,20,RED); //画点 1
71.              TP_Adj_Info_Show(pos_temp[0][0],pos_temp[0][1],pos_temp[1][0],pos_t
emp[1][1],pos_temp[2][0],pos_temp[2][1],pos_temp[3][0],pos_temp[3][1],fac*100); //显示数据
72.              continue;
73.          }
74.          tem1=abs(pos_temp[0][0]-pos_temp[2][0]); //x1-x3
75.          tem2=abs(pos_temp[0][1]-pos_temp[2][1]); //y1-y3
76.          tem1*=tem1;
77.          tem2*=tem2;
78.          d1=sqrt(tem1+tem2); //得到 1,3 的距离
79.
80.          tem1=abs(pos_temp[1][0]-pos_temp[3][0]); //x2-x4
81.          tem2=abs(pos_temp[1][1]-pos_temp[3][1]); //y2-y4
82.          tem1*=tem1;
83.          tem2*=tem2;
84.          d2=sqrt(tem1+tem2); //得到 2,4 的距离
85.          fac=(float)d1/d2;
86.          if(fac<0.95||fac>1.05) //不合格
87.          {
88.              cnt=0;
89.              TP_Drow_Touch_Point(tftlcd_data.width-20,tftlcd_data.height-20,WHIT
E); //清除点 4

```

```

90.                TP_Drow_Touch_Point(20,20,RED);    //画点 1
91.                TP_Adj_Info_Show(pos_temp[0][0],pos_temp[0][1],pos_temp[1][0],pos_t
emp[1][1],pos_temp[2][0],pos_temp[2][1],pos_temp[3][0],pos_temp[3][1],fac*100);//显示数据
92.                continue;
93.            }//正确了
94.
95.            //对角线相等
96.            tem1=abs(pos_temp[1][0]-pos_temp[2][0]); //x1-x3
97.            tem2=abs(pos_temp[1][1]-pos_temp[2][1]); //y1-y3
98.            tem1*=tem1;
99.            tem2*=tem2;
100.           d1=sqrt(tem1+tem2);//得到 1,4 的距离
101.
102.           tem1=abs(pos_temp[0][0]-pos_temp[3][0]); //x2-x4
103.           tem2=abs(pos_temp[0][1]-pos_temp[3][1]); //y2-y4
104.           tem1*=tem1;
105.           tem2*=tem2;
106.           d2=sqrt(tem1+tem2);//得到 2,3 的距离
107.           fac=(float)d1/d2;
108.           if(fac<0.95||fac>1.05)//不合格
109.           {
110.               cnt=0;
111.               TP_Drow_Touch_Point(tftlcd_data.width-20,tftlcd_data.height-20,WHIT
E); //清除点 4
112.               TP_Drow_Touch_Point(20,20,RED);    //画点 1
113.               TP_Adj_Info_Show(pos_temp[0][0],pos_temp[0][1],pos_temp[1][0],pos_t
emp[1][1],pos_temp[2][0],pos_temp[2][1],pos_temp[3][0],pos_temp[3][1],fac*100);//显示数据
114.               continue;
115.           }//正确了
116.           //计算结果
117.           tp_dev.xfac=(float)(tftlcd_data.width-40)/(pos_temp[1][0]-pos_temp[0][0]
); //得到 xfac
118.           tp_dev.xoff=(tftlcd_data.width-tp_dev.xfac*(pos_temp[1][0]+pos_temp[0][
0]))/2; //得到 xoff
119.
120.           tp_dev.yfac=(float)(tftlcd_data.height-40)/(pos_temp[2][1]-pos_temp[0][
1]); //得到 yfac
121.           tp_dev.yoff=(tftlcd_data.height-tp_dev.yfac*(pos_temp[2][1]+pos_temp[0]
[1]))/2; //得到 yoff
122.           if(abs(tp_dev.xfac)>2||abs(tp_dev.yfac)>2)//触屏和预设的相反了.
123.           {
124.               cnt=0;
125.               TP_Drow_Touch_Point(tftlcd_data.width-20,tftlcd_data.height-20,WHIT
E); //清除点 4

```

```

126.                TP_Drow_Touch_Point(20,20,RED);           //画点 1
127.                LCD_ShowString(40,26,tftlcd_data.width,tftlcd_data.height,16,"TP Ne
    ed readjust!");
128.                tp_dev.touchtype=!tp_dev.touchtype;//修改触屏类型.
129.                if(tp_dev.touchtype)//X,Y 方向与屏幕相反
130.                {
131.                    CMD_RDY=0XD0;
132.                    CMD_RDY=0XD0;
133.                }else                                     //X,Y 方向与屏幕相同
134.                {
135.                    CMD_RDY=0XD0;
136.                    CMD_RDY=0XD0;
137.                }
138.                continue;
139.            }
140.            FRONT_COLOR=BLUE;
141.            LCD_Clear(WHITE);//清屏
142.            LCD_ShowString(35,110,tftlcd_data.width,tftlcd_data.height,16,"Touch Sc
    reen Adjust OK!");//校正完成
143.            delay_ms(1000);
144.            TP_Save_Adjdata();
145.            LCD_Clear(WHITE);//清屏
146.            return;//校正完成
147.        }
148.    }
149.    delay_ms(10);
150.    outtime++;
151.    if(outtime>1000)
152.    {
153.        TP_Get_Adjdata();
154.        break;
155.    }
156.    }
157. #endif
158.
159. #if defined(TFTLCD_NT35510)
160.     return; //电容触摸屏不需要校准
161. #endif
162. }

```

TP_Adjust 函数是触摸实验最核心的代码，在这里给大家介绍一下我们这里所使用的触摸屏校正原理：我们传统的鼠标是一种相对定位系统，只和上一次鼠标的位置坐标有关。而触摸屏则是一种绝对坐标系统，要选哪就直接点哪，与相

对定位系统有着本质的区别。绝对坐标系统的特点是每一次定位坐标与上一次定位坐标没有关系，每次触摸的数据通过校准转为屏幕上的坐标，不管在什么情况下，触摸屏这套坐标在同一点的输出数据是稳定的。不过由于技术原理的原因，并不能保证同一点触摸每一次采样数据相同，不能保证绝对坐标定位，点不准，这就是触摸屏最怕出现的问题：漂移。对于性能质量好的触摸屏来说，漂移的情况出现并不是很严重。所以很多应用触摸屏的系统启动后，进入应用程序前，先要执行校准程序。通常应用程序中使用的 LCD 坐标是以像素为单位的。比如说：左上角的坐标是一组非 0 的数值，比如（20，20），而右下角的坐标为（220，300）。这些点的坐标都是以像素为单位的，而从触摸屏中读出的是点的物理坐标，其坐标轴的方向、XY 值的比例因子、偏移量都与 LCD 坐标不同，所以，需要在程序中把物理坐标首先转换为像素坐标，然后再赋给 POS 结构，达到坐标转换的目的。

校正思路：在了解了校正原理之后，我们可以得出下面的一个从物理坐标到像素坐标的转换关系式：

$$\begin{aligned} \text{LCDx} &= \text{xfac} * \text{Px} + \text{xoff}; \\ \text{LCDy} &= \text{yfac} * \text{Py} + \text{yoff}; \end{aligned}$$

其中 (LCDx, LCDy) 是在 LCD 上的像素坐标，(Px, Py) 是从触摸屏读到的物理坐标。xfac, yfac 分别是 X 轴方向和 Y 轴方向的比例因子，而 xoff 和 yoff 则是这两个方向的偏移量。

这样我们只要事先在屏幕上面显示 4 个点（这四个点的坐标是已知的），分别按这四个点就可以从触摸屏读到 4 个物理坐标，这样就可以通过待定系数法求出 xfac、yfac、xoff、yoff 这四个参数。我们保存好这四个参数，在以后的使用中，我们把所有得到的物理坐标都按照这个关系式来计算，得到的就是准确的屏幕坐标，达到了触摸屏校准的目的。

该函数内调用了比较关键的函数 TP_Scan，用来读取触摸屏 X\Y 轴的物理坐标，因为只有获取物理坐标后我们才能转换对应 LCD 的坐标，这个在前面已介绍。TP_Scan 函数代码如下：

1. //触摸按键扫描
2. //tp:0,屏幕坐标;1,物理坐标(校准等特殊场合用)
3. //返回值:当前触屏状态.
4. //0,触屏无触摸;1,触屏有触摸
5. u8 TP_Scan(u8 tp)

```

6.  {
7.  #if defined(TFTLCD_HX8357D)||defined(TFTLCD_ILI9341)||defined(TFTLCD_HX8352C)|| \
8.      defined(TFTLCD_R61509V)||defined(TFTLCD_R61509VN)||defined(TFTLCD_R61509V3)|| \
9.      defined(TFTLCD_ILI9486)||defined(TFTLCD_ST7793)||defined(TFTLCD_ILI9325)|| \
10.     defined(TFTLCD_ILI9327)||defined(TFTLCD_ILI9481)||defined(TFTLCD_SSD1963)
11.
12.     if(PEN==0)//有按键按下
13.     {
14.         if(tp)TP_Read_XY2(&tp_dev.x[0],&tp_dev.y[0]);//读取物理坐标
15.         else if(TP_Read_XY2(&tp_dev.x[0],&tp_dev.y[0]))//读取屏幕坐标
16.         {
17.             tp_dev.x[0]=tp_dev.xfac*tp_dev.x[0]+tp_dev.xoff;//将结果转换为屏幕坐标
18.             tp_dev.y[0]=tp_dev.yfac*tp_dev.y[0]+tp_dev.yoff;
19.         }
20.         if((tp_dev.sta&TP_PRES_DOWN)==0)//之前没有被按下
21.         {
22.             tp_dev.sta=TP_PRES_DOWN|TP_CATH_PRES;//按键按下
23.             tp_dev.x[4]=tp_dev.x[0];//记录第一次按下时的坐标
24.             tp_dev.y[4]=tp_dev.y[0];
25.         }
26.     }else
27.     {
28.         if(tp_dev.sta&TP_PRES_DOWN)//之前是被按下的
29.         {
30.             tp_dev.sta&=~(1<<7);//标记按键松开
31.         }else//之前就没有被按下
32.         {
33.             tp_dev.x[4]=0;
34.             tp_dev.y[4]=0;
35.             tp_dev.x[0]=0xffff;
36.             tp_dev.y[0]=0xffff;
37.         }
38.     }
39.     return tp_dev.sta&TP_PRES_DOWN;//返回当前的触屏状态
40. #endif
41.
42. #if defined(TFTLCD_NT35510)
43.     return (!GT5663_Scan(0));
44. #endif
45. }

```

函数内主要通过 TP_Read_XY2 函数获取物理坐标，该函数代码如下：

```

1.  //读取 x,y 坐标
2.  //最小值不能少于 100.
3.  //x,y: 读取到的坐标值

```



```

4.    //返回值:0,失败;1,成功。
5.    u8 TP_Read_XY(u16 *x,u16 *y)
6.    {
7.        u16 xtemp,ytemp;
8.        xtemp=TP_Read_XOY(CMD_RDX);
9.        ytemp=TP_Read_XOY(CMD_RDY);
10.        //if(xtemp<100||ytemp<100)return 0;//读数失败
11.        *x=xtemp;
12.        *y=ytemp;
13.        return 1;//读数成功
14.    }
15.    //连续 2 次读取触摸屏 IC,且这两次的偏差不能超过
16.    //ERR_RANGE,满足条件,则认为读数正确,否则读数错误。
17.    //该函数能大大提高准确度
18.    //x,y:读取到的坐标值
19.    //返回值:0,失败;1,成功。
20.    #define ERR_RANGE 50 //误差范围
21.    u8 TP_Read_XY2(u16 *x,u16 *y)
22.    {
23.        u16 x1,y1;
24.        u16 x2,y2;
25.        u8 flag;
26.        flag=TP_Read_XY(&x1,&y1);
27.        if(flag==0)return(0);
28.        flag=TP_Read_XY(&x2,&y2);
29.        if(flag==0)return(0);
30.        if(((x2<=x1&&x1<x2+ERR_RANGE)|| (x1<=x2&&x2<x1+ERR_RANGE))//前后两次采样在+-50 内
31.        &&((y2<=y1&&y1<y2+ERR_RANGE)|| (y1<=y2&&y2<y1+ERR_RANGE)))
32.        {
33.            *x=(x1+x2)/2;
34.            *y=(y1+y2)/2;
35.            return 1;
36.        }else return 0;
37.    }

```

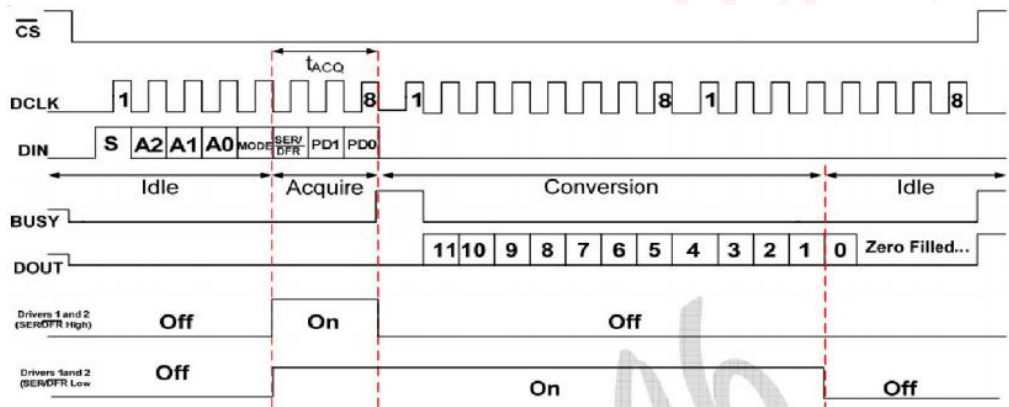
该函数首先通过 TP_Read_XOY 函数获取触摸屏物理坐标值,然后通过相应的程序滤波,保证数据的准确性,防止出现飞点等误差。比较常用的程序滤波的方法其实就是多次数据的读取,然后把最大最小值除去,算出平均值。这种方法读取的次数越多,得到的数据就越准确。不过为了更好的滤波,还使用了另外一种方式进行滤波,也就是当读取到两次数据之后,然后检查两个数据之间的差值,如果超过理想的误差,那么丢弃数据。这种方法也是处理飞点现象的常用方法。

该函数里面用到了很多的宏,比如 CMD_RDX、CMD_RDY 等,这些都在 touch.c

文件内定义了。CMD_RDY=0xD0 和 CMD_RDY=0x90 是 XPT2046 AD 芯片读取 X 和 Y 轴的命令，可通过《XPT2046》芯片数据手册查找到，如下图：

A2	A1	A0	VBAT	AUXIN	TEMP	YN	XP	YP	Y-位置	X-位置	Z1-位置	Z2-位置	X-驱动	Y-驱动
0	0	0			+IN (TEMP0)								off	off
0	0	1					+IN		测量				Off	On
0	1	0	+IN										Off	Off
0	1	1					+IN				测量		XN, On	YP, On
1	0	0				+IN					测量		XN, On	YP, On
1	0	1						+IN		测量			On	Off
1	1	0		+IN									Off	Off
1	1	1			+IN (TEMP1)								Off	Off

这里要特别说明下，要读取 XPT2046 的数据，根据其时序图可知，时序图如下：



XPT2046 完成一个完整的转换需要 24 个串行时钟，也就是需要 3 个字节的 SPI 时钟。对照上图，XPT2046 前 8 个串行时钟，是接收 1 个字节的转换命令，接收到转换命令了之后，然后使用 1 个串行时钟的时间来完成数据转换（当然在编写程序的时候，为了得到精确的数据，你可以适当的延时一下），然后返回 12 个字节长度（12 个字节长度也计时 12 个串行时钟）的转换结果。然后最后 4 个串行时钟返回 4 个无效数据。这一过程在 TP_Read_AD 函数内实现。代码如下：

```
1. //SPI 读数据
2. //从触摸屏 IC 读取 adc 值
3. //CMD:指令
4. //返回值:读到的数据
5. u16 TP_Read_AD(u8 CMD)
6. {
7.     u8 count=0;
8.     u16 Num=0;
9.     TCLK=0;    //先拉低时钟
```

```

10.     TDIN=0;      //拉低数据线
11.     TCS=0;       //选中触摸屏 IC
12.     TP_Write_Byte(CMD); //发送命令字
13.     delay_us(6); //ADS7846 的转换时间最长为 6us
14.     TCLK=0;
15.     delay_us(1);
16.     TCLK=1;      //给 1 个时钟，清除 BUSY
17.     delay_us(1);
18.     TCLK=0;
19.     for(count=0;count<16;count++)//读出 16 位数据,只有高 12 位有效
20.     {
21.         Num<=1;
22.         TCLK=0; //下降沿有效
23.         delay_us(1);
24.         TCLK=1;
25.         if(DOUT)Num++;
26.     }
27.     Num>>=4;      //只有高 12 位有效.
28.     TCS=1;       //释放片选
29.     return(Num);
30. }

```

3.3.3 触摸屏扫描函数

要检测是否有触摸，我们可以编写一个触摸扫描函数，并在该函数中获取触摸屏的物理坐标和 LCD 坐标。具体代码如下：

```

1.     //触摸按键扫描
2.     //tp:0,屏幕坐标;1,物理坐标(校准等特殊场合用)
3.     //返回值:当前触屏状态.
4.     //0,触屏无触摸;1,触屏有触摸
5.     u8 TP_Scan(u8 tp)
6.     {
7.         #if defined(TFTLCD_HX8357D) || defined(TFTLCD_ILI9341) || defined(TFTLCD_HX8352C) || \
8.             defined(TFTLCD_R61509V) || defined(TFTLCD_R61509VN) || defined(TFTLCD_R61509V3) || \
9.             defined(TFTLCD_ILI9486) || defined(TFTLCD_ST7793) || defined(TFTLCD_ILI9325) || \
10.            defined(TFTLCD_ILI9327) || defined(TFTLCD_ILI9481) || defined(TFTLCD_SSD1963)
11.
12.         if(PEN==0)//有按键按下
13.         {
14.             if(tp)TP_Read_XY2(&tp_dev.x[0],&tp_dev.y[0]); //读取物理坐标
15.             else if(TP_Read_XY2(&tp_dev.x[0],&tp_dev.y[0]))//读取屏幕坐标
16.             {
17.                 tp_dev.x[0]=tp_dev.xfac*tp_dev.x[0]+tp_dev.xoff; //将结果转换为屏幕坐标

```

```

18.         tp_dev.y[0]=tp_dev.yfac*tp_dev.y[0]+tp_dev.yoff;
19.     }
20.     if((tp_dev.sta&TP_PRES_DOWN)==0)//之前没有被按下
21.     {
22.         tp_dev.sta=TP_PRES_DOWN|TP_CATH_PRES;//按键按下
23.         tp_dev.x[4]=tp_dev.x[0];//记录第一次按下时的坐标
24.         tp_dev.y[4]=tp_dev.y[0];
25.     }
26. }else
27. {
28.     if(tp_dev.sta&TP_PRES_DOWN)//之前是被按下的
29.     {
30.         tp_dev.sta&=~(1<<7);//标记按键松开
31.     }else//之前就没有被按下
32.     {
33.         tp_dev.x[4]=0;
34.         tp_dev.y[4]=0;
35.         tp_dev.x[0]=0xffff;
36.         tp_dev.y[0]=0xffff;
37.     }
38. }
39. return tp_dev.sta&TP_PRES_DOWN;//返回当前的触屏状态
40. #endif
41.
42. #if defined(TFTLCD_NT35510)
43.     return (!GT5663_Scan(0));
44. #endif
45. }

```

该函数功能比较简单，函数有一个入口参数，用于选择是获取物理坐标还是 LCD 实际坐标；函数有一个返回值，用于判定是否有触摸，如果为 1 表示有触摸按下，否则无触摸。里面调用的函数 TP_Read_XY2 在前面都已介绍，将获取到的物理坐标存储在结构体 tp_dev.x/tp_dev.y 中，然后根据 LCD 实际坐标转换公式求取 LCD 实际坐标。涉及到的结构体在 touch.h 文件中已经定义为全局变量。

在该函数中，根据 LCD 不同型号来判断是电阻触摸还是电容触摸，如果是电容触摸屏调用了 GT5663_Scan 函数来扫描电容触摸，该函数代码如下：

```

1.     const u16 GT5663_TPX_TBL[5]={GT_TP1_REG,GT_TP2_REG,GT_TP3_REG,GT_TP4_REG,GT_TP5_REG};
2.     //扫描触摸屏(采用查询方式)
3.     //mode:0,正常扫描.
4.     //返回值:当前触屏状态.
5.     //1,触屏无触摸;0,触屏有触摸
6.     u8 GT5663_Scan(u8 mode)

```

```

7.  {
8.      u8 buf[4];
9.      u8 i=0;
10.     u8 res=1;
11.     u8 temp;
12.     u8 tp_sta;
13.     static u8 t=0; //控制查询间隔,从而降低 CPU 占用率
14.     t++;
15.     if((t%10)==0||t<10) //空闲时,每进入 10 次 CTP_Scan 函数才检测 1 次,从而节省 CPU 使用率
16.     {
17.         GT5663_RD_Reg(GT_GSTID_REG,&mode,1);    //读取触摸点的状态
18.         // printf("mode=%d\r\n",mode);
19.         if(mode&0X80&&((mode&0XF)<6))
20.         {
21.             temp=0;
22.             GT5663_WR_Reg(GT_GSTID_REG,&temp,1); //清标志
23.         }
24.         if((mode&0XF)&&((mode&0XF)<6))
25.         {
26.             temp=0XFF<<(mode&0XF);    //将点的个数转换为 1 的位数,匹配 tp_dev.sta 定义
27.             tp_dev.sta=(~temp);
28.             tp_dev.x[4]=tp_dev.x[0];    //保存触点 0 的数据
29.             tp_dev.y[4]=tp_dev.y[0];
30.             for(i=0;i<5;i++)
31.             {
32.                 if(tp_dev.sta&(1<<i)) //触摸有效?
33.                 {
34.                     GT5663_RD_Reg(GT5663_TPX_TBL[i],buf,4); //读取 XY 坐标值
35.                     if(tftlcd_data.dir==1) //横屏
36.                     {
37.                         tp_dev.y[i]=((u16)buf[1]<<8)+buf[0];
38.                         tp_dev.x[i]=800-(((u16)buf[3]<<8)+buf[2]);
39.                     }else
40.                     {
41.                         tp_dev.x[i]=((u16)buf[1]<<8)+buf[0];
42.                         tp_dev.y[i]=((u16)buf[3]<<8)+buf[2];
43.                     }
44.                     //printf("x[%d]:%d,y[%d]:%d\r\n",i,TouchData.lcdx[i],i,TouchData.lcdy[i]
45.                 );
46.             }
47.             res=0;
48.             if(tp_dev.x[0]>tftlcd_data.width||tp_dev.y[0]>tftlcd_data.height) //非法数据(坐标
               超出了)

```

```

49.         {
50.             if((mode&0XF)>1)    //有其他点有数据,则复第二个触点的数据到第一个触点.
51.             {
52.                 tp_dev.x[0]=tp_dev.x[1];
53.                 tp_dev.y[0]=tp_dev.y[1];
54.                 t=0;            //触发一次,则会最少连续监测 10 次,从而提高命中率
55.             }else              //非法数据,则忽略此次数据(还原原来的)
56.             {
57.                 tp_dev.x[0]=tp_dev.x[4];
58.                 tp_dev.y[0]=tp_dev.y[4];
59.                 mode=0X80;
60.             }
61.         }else t=0;            //触发一次,则会最少连续监测 10 次,从而提高命中率
62.     }
63. }
64. if((mode&0X8F)==0X80)//无触摸点按下
65. {
66.     tp_dev.x[0]=0xffff;
67.     tp_dev.y[0]=0xffff;
68.     tp_dev.sta=0;    //清除点有效标记
69. }
70. if(t>240)t=10;//重新从 10 开始计数
71. return res;
72. }

```

该函数用于扫描电容触摸屏是否有按键下,由我们是采用查询的方式读取数据,所以这里使了一个静态变量(static)来提高效率当无触摸时候尽减少对 CPU 的占用,当有触摸时候又保证能迅速检测到。读取数据时,先读取状态寄存器(GT_GSTID_REG)的值,从而判断触摸点的个数(最多 10 个),然后依次读取各触摸点的坐标数据,在读取到数据后,还需要根据屏幕的分辨率和横竖屏状态进行坐标变换。另外,在遇到非法数据的时候,需要对非法数据进行处理,以免干扰程序的正常运行。对于函数内所调用的其他功能函数大家可以参考源码,这里就不列出来。

3.3.4 主函数

编写好触摸屏初始化、触摸校准和扫描函数后,接下来就可以编写主函数了,代码如下:

```

1.  #include "system.h"
2.  #include "SysTick.h"

```

```

3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "touch.h"
8.
9.
10. void kai_display() //开机显示
11. {
12.     FRONT_COLOR=BLACK;
13.     LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"Touch Test");
14.     LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.net");
15.     LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,16,"K_UP:Adjust");
16. }
17.
18. void display_init() //初始化显示
19. {
20.     FRONT_COLOR=RED;
21.     LCD_ShowString(tftlcd_data.width-8*4,0,tftlcd_data.width,tftlcd_data.height,16,"RST");
22.     LCD_Fill(120, tftlcd_data.height - 16, 139, tftlcd_data.height, BLUE);
23.     LCD_Fill(140, tftlcd_data.height - 16, 159, tftlcd_data.height, RED);
24.     LCD_Fill(160, tftlcd_data.height - 16, 179, tftlcd_data.height, MAGENTA);
25.     LCD_Fill(180, tftlcd_data.height - 16, 199, tftlcd_data.height, GREEN);
26.     LCD_Fill(200, tftlcd_data.height - 16, 219, tftlcd_data.height, CYAN);
27.     LCD_Fill(220, tftlcd_data.height - 16, 239, tftlcd_data.height, YELLOW);
28. }
29.
30. int main()
31. {
32.     u8 i=0;
33.     u8 key;
34.     u16 penColor = BLUE;
35.
36.     SysTick_Init(72);
37.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
38.     LED_Init();
39.     USART1_Init(115200);
40.     TFTLCD_Init(); //LCD 初始化
41.     KEY_Init();
42.     TP_Init();
43.
44.     kai_display();
45.     delay_ms(2000);

```



```
46.     LCD_Clear(WHITE);
47.     display_init();
48.
49.     while(1)
50.     {
51.         key=KEY_Scan(0);
52.         if(key==KEY_UP_PRESS)
53.         {
54.             TP_Adjust(); //校正
55.             display_init();
56.         }
57.         if(TP_Scan(0))
58.         {
59.             /* 选择画笔的颜色 */
60.             if(tp_dev.y[0] > tftlcd_data.height - 18&&tp_dev.y[0]<tftlcd_data.height)
61.             {
62.                 if(tp_dev.x[0]>220)
63.                 {
64.                     penColor = YELLOW;
65.                 }
66.                 else if(tp_dev.x[0]>200)
67.                 {
68.                     penColor = CYAN;
69.                 }
70.
71.                 else if(tp_dev.x[0]>180)
72.                 {
73.                     penColor = GREEN;
74.                 }
75.                 else if(tp_dev.x[0]>160)
76.                 {
77.                     penColor = MAGENTA;
78.                 }
79.                 else if(tp_dev.x[0]>140)
80.                 {
81.                     penColor = RED;
82.
83.                 }
84.                 else if(tp_dev.x[0]>120)
85.                 {
86.                     penColor = BLUE;
87.                 }
88.             }
89.             else //画点
```

```

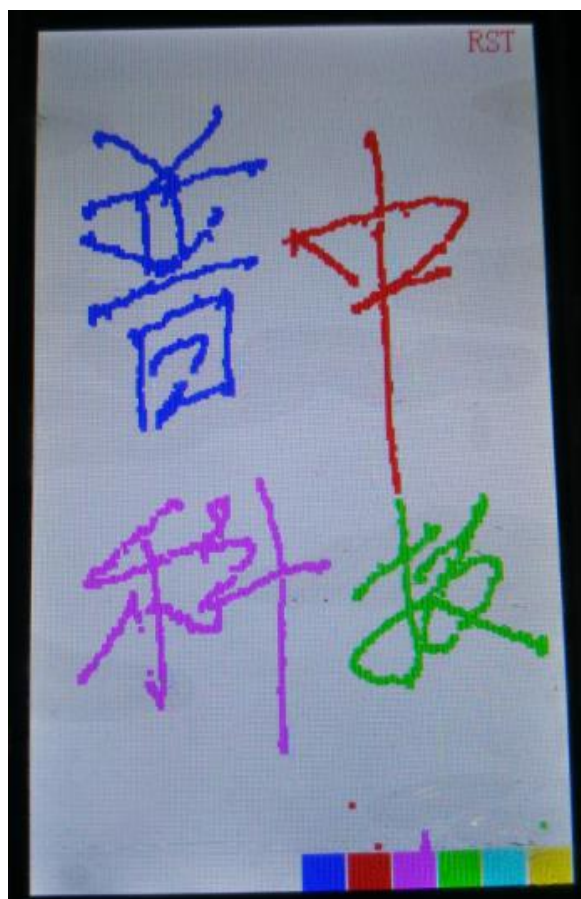
90.         {
91.             LCD_Fill(tp_dev.x[0]-1, tp_dev.y[0]-1, tp_dev.x[0]+2,
92.                 tp_dev.y[0]+2, penColor);
93.         }
94.
95.         /* 清屏 */
96.         if ((tp_dev.x[0] > tftlcd_data.width-8*4) && (tp_dev.y[0] < 16))
97.         {
98.             LCD_Fill(0, 0, tftlcd_data.width-1,tftlcd_data.height-16-1, BACK_COLOR);
99.             LCD_ShowString(tftlcd_data.width-8*4,0,tftlcd_data.width,tftlcd_data.height,
16, "RST");
100.        }
101.    }
102.
103.    i++;
104.    if(i%20==0)
105.    {
106.        LED1=!LED1;
107.    }
108.    //    delay_ms(10);
109.    }
110. }

```

主函数实现的功能很简单，首先调用之前编写好的硬件初始化函数，包括 SysTick 系统时钟，LED 初始化等。然后调用我们前面编写的 TP_Init 触摸屏初始化函数，如果未校准的首先会进行校准，接着进入 while 循环，调用 KEY_Scan 函数，不断检测 KEY_UP 键是否按下，如果按下进行触摸屏校准（电阻屏校准，电容屏无需校准），然后调用 TP_Scan 触摸扫描函数判断是否有触摸，并获取触摸屏 LCD 坐标，同时触摸后描出对应的点，最后控制 DS0 指示灯间隔 200ms 闪烁，提示系统正常运行。

3.4 实验现象

将工程程序编译后下载到开发板内，可以看到 DS0 指示灯不断闪烁，表示程序正常运行。触摸屏显示界面如下图所示：



我们可以在电阻屏上画一些内容，屏幕右上角的 RST 可以用来清屏，只要触摸下 RST 这个区域即可将屏幕上所画的内容擦除掉。如果出现触摸不准，可以按下 KEY_UP 键，进入触摸校准界面，按照提示校准下即可。

课后作业

(1) 在触摸屏上设计 2 个触摸按钮，控制开发板上蜂鸣器的开关。（温馨提示：使用 LCD API 函数画 2 个按键，通过判断这两个按键的位置坐标实现控制）